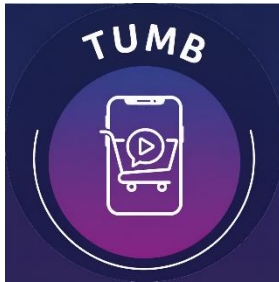


Taibah University Marketplace Bot (TUMB)



Graduation Project

by

هشام محمد حمد الأحمدى	4420080
بندر ملىح مخيضير العوفى	4421064
عمار ياسر عواد المغامسى	4303002
يوسف ناجى القرافى	4303056
نواف ربيعان الحجيلى	4201347
على عبدالله الشاطرى	4307234
باسل فهد الحجيلى	4102768
حمزة محمد احمد فال	4108423

**A project submitted in partial fulfilment of the requirements
for the degree of Bachelor of Science (Computer Science)**

Supervised by

Dr. Hamza Ghandorh

1st Semester - Academic Year 1447 (2025)

Declaration of Authorship and AI Usage

Report Title: Taibah University Marketplace Bot

Date: 2026/5/14

We, the undersigned, hereby declare that this report is our original work and has been composed in accordance with the academic integrity policies of Taibah University. I confirm that all sources of information, including data, concepts, and direct quotations, have been properly cited and acknowledged in the bibliography.

In the preparation of this document,

- ☐ I have not used any generative AI tools.
- ☒ I have utilized generative AI tools solely for the purpose of [Choose one or more of the following and be specific:
 - ☒ Brainstorming and structuring initial ideas.
 - ☒ Refining grammar and syntax.
 - ☒ Generating example code snippets for illustrative purposes.
 - ☒ Summarizing a body of text to create an abstract or introduction.
 - ☒ Creating a foundational outline for the report's sections.].

The content generated by these AI tools has been thoroughly reviewed, verified, and edited by us to ensure its accuracy, relevance, and alignment with the report's objectives. I assume full responsibility for the content, arguments, and conclusions presented herein.

We affirm that the core research, analysis, and original thoughts presented in this report are entirely our own intellectual contribution. Any and all use of generative AI has been explicitly disclosed in the footnotes or a dedicated section within this report, where applicable.

By submitting this report, we acknowledge the importance of transparency and intellectual honesty in the evolving landscape of digital and automated tools.

Signature:

[هشام محمد الأحمدى، بندر ملىح العوفى، عمار ياسر المغماسى، يوسف ناجى القرافى، نواف ربيعان الحبلى، على
عبد الله الشاطرى، باسل فهد الحبلى، حمزة محمد فال]

Abstract

Students at Taibah University face challenges finding affordable and secure ways to buy and sell academic supplies on campus. Our project aims to offer new shopping experiences for TU students and staff, promotes economic efficiency, and supports a sustainable circular economy. The platform is a rule-based conversational marketplace that lets users list, search, and compare items through a chatbot interface, providing a trusted and efficient e-commerce solution for the university community.

Keywords: Chatbot; Marketplace; Rule-Based System; Conversational
Interface; E-commerce

Acknowledgement

We would like to express our sincere gratitude to our supervisor, Dr. Hamza Ghandorh, for his continuous support, guidance, and valuable insights throughout the development of this project. His expertise and encouragement were instrumental in shaping the direction and quality of our work. We also extend our appreciation to the faculty of the College of Computer Science and Engineering at Taibah University for providing us with the resources and academic environment necessary to complete this project successfully. Finally, we thank our families and colleagues for their patience, motivation, and unwavering support during this journey

Table of Content

Declaration of Authorship and AI Usage.....	ii
Abstract.....	iii
Acknowledgement	iv
Table of Content.....	v
List of Figures:	x
List of Tables:.....	xii
List of Abbreviations	xiv
1 Chapter 1: Introduction.....	1
1.1 Introduction	1
1.2 Problem Definition:	1
1.3 Project Objectives	2
1.4 Project Scope.....	2
1.5 Contributions of This Study:	2
1.6 Project Timeline	3
1.7 Document Organization	3
2 Chapter 2: Literature Review	5
2.1 Introduction	5
2.2 Background	5
2.2.1 Marketplace and Second-Hand Goods.....	5
2.2.2 Usability and Accessibility:	5
2.2.3 Payment Gateways:.....	6
2.2.4 Shipping and Logistics:.....	6
2.2.5 Rule Engine:.....	6
2.2.6 User Intent Recognition:	7
2.2.7 Dialog Management:.....	7

2.2.8 Response Generation:	7
2.2.9 Context Handling:	8
2.2.10 Peer-to-Peer (P2P) Exchange:	8
2.2.11 Circular Economy and Resource Reuse:	8
2.2.12 Conversational Interface:	9
2.2.13 Artificial Intelligence and Rule-Based Systems:	9
2.2.14 Software Development Stack:	10
2.2.15 A UI/UX Design:	12
2.2.16 API and CRUD Operations:	13
2.2.17 Database Management and Mobile Development:	13
2.2.18 Mobile Development:	13
2.2.19 UML Modeling Tools:	14
2.3 Related Work:	15
2.3.1 Review of Relevant Work:	15
2.3.2 Relationship Between Relevant Work and Our Own Work:	26
2.4 Summary:	27
Chapter 3: System Analysis	28
3.1 Introduction	28
3.2 Requirements Elicitation	28
3.2.1 Functional Requirements	28
3.2.2 Non-Functional Requirements	29
3.2.3 User Requirements or Domain Requirements	30
3.3 Requirements Specification	32
3.4 Developmental Methodology	42
3.4.1 Waterfall Model-Design	43
3.4.2 Advantages and Disadvantages of the Waterfall Model	43

3.4.4 Constraints and Resources	46
3.5 Data Collecting and Processing	46
3.6 Summary	47
Chapter4: System Design	48
4.1 Introduction.....	48
4.2 Architectural Design	48
4.3 Object Oriented Design.....	49
4.3.1 Structural Static Models.....	49
4.3.2 Dynamic Models	49
4.4 Data Modeling	55
4.4.1 Database Design.....	55
4.4.2 Relational Schema	55
4.4.3 Database Table.....	56
4.5 User Interface Design	59
4.6 Summary.....	71
Chapter 5: System Implementation	72
5.1 Introduction.....	72
5.2 Tools and Languages	72
5.3 Mapping Design to Implementation	73
5.3.1 Client–Server Architecture Implementation	73
5.3.2 API Layer Implementation	73
5.3.3 Application Layer Implementation	73
5.3.4 Domain Layer (Rule Engine).....	73
5.3.5 Persistence Layer Implementation.....	74
5.4 Main/Most Important Codes	74
5.5 System testing	81

5.5.1 Unit test.....	81
Chapter 6: Case-study: AI Agent Extension	87
6.1 Motivation and Objectives	Error! Bookmark not defined.
6.2 Overview of Added Technologies.....	Error! Bookmark not defined.
6.2.1 Large Language Models (LLMs).....	88
6.2.2 The Model Context Protocol (MCP).....	88
6.2.3 LangChain and LangGraph.....	88
6.3 System Architecture.....	89
6.4 MCP Server Implementation	90
6.4.1 Tool Declaration.....	90
6.4.2 Tool Dispatch.....	92
6.5 LLM Agent Implementation	93
6.5.1 Initialisation	93
6.6 Conversation Flow Walkthrough.....	98
6.7 Comparative Analysis: Rule-Based vs. AI-Powered Agent	99
6.8 Advantages of the AI-Powered Approach	100
6.8.1 Robustness to Linguistic Variation.....	100
6.8.2 Autonomous Multi-Step Reasoning.....	100
6.8.3 Reduced Development Effort for New Features.....	101
6.8.4 Backward Compatibility	101
6.8.5 Standards-Based Tool Integration.....	101
6.9 Limitations and Challenges.....	101
6.9.1 Latency.....	101
6.9.2 Operational Cost	102
6.9.3 Non-Determinism	102
6.9.4 Prompt Injection and Adversarial Inputs	102

6.9.5 Statelessness and Context Window Limits	102
6.10 Security Considerations	103
6.11 Summary	103
Chapter 7: Conclusion.....	104
7.1 Goals Achieved.....	104
7.2 Limitations and Future Work.....	105
4 References	106

List of Figures:

Figure 1 Gantt Chart	3
Figure 2 Artificial Intelligence and Rule-Based Systems.....	9
Figure 3 Haraj	16
Figure 4 Facebook marketplace	18
Figure 5 TreasureX	20
Figure 6 KFUPM Store.....	22
Figure 7 KFUPM telegram market	24
Figure 8 TU telegram group for second-hand market	24
Figure 9 Requirements Specification.....	32
Figure 10 Waterfall Model.....	43
Figure 11 Architectural Diagram	48
Figure 12 Class Diagram	49
Figure 13 Admin Sequence Diagram.....	50
Figure 14 User Sequence Diagram	51
Figure 15 Post Sequence Diagram.....	52
Figure 16 Search Sequence Diagram.....	53
Figure 17 Monitoring Sequence Diagram.....	54
Figure 18 ER Diagram	55
Figure 19 Relational schema.....	56
Figure 20 Login and Registration	59
Figure 21 Chat.....	60
Figure 22 Reports.....	61
Figure 23 Mange Posts (reports).....	62
Figure 24 Admin Management	63
Figure 25 User & Admin Profile	64
Figure 26 Admin Dashboard.....	65
Figure 27 Post & Search	66
Figure 28 Buy	67
Figure 29 Compare	68
Figure 30 Mange	69

Figure 31 Recommendation.....	70
Figure 32 Register User Function	74
Figure 33 Login User Function.....	75
Figure 34 Create Post Function.....	75
Figure 35 Compare Post Function	76
Figure 36 Search Post Function	77
Figure 37 Report Post Function	78
Figure 38 Update Post.....	78
Figure 39 Get Recommendations.....	79
Figure 40 Create Request.....	80
Figure 41 Client-server architecture of the AI-powered TUMB system	90
Figure 42 Tool declaration using mcp.types.....	92
Figure 43 Tool dispatch logic	93
Figure 44 Agent initialization	94
Figure 45 System prompt template in llm_agent.py	95
Figure 46 The run_agent() entry point.....	96
Figure 47 Tool-to-action mapping and structured data.....	97
Figure 48 Live conversation demonstrating the full request-response cycle.....	99

List of Tables:

Table 1 UML Modeling Tools	15
Table 2 Comparison with Haraj Platform	17
Table 3 Comparison with Facebook Platform	19
Table 4 Comparison with TreasureX Platform	21
Table 5 Comparison with KFUPM Store Platform.....	23
Table 6 Comparison with Telegram Groups.....	26
Table 7 Relationship Between Works.....	27
Table 8 Non-Functional Requirements	30
Table 9 Register Use Case	33
Table 10 Login Use Case	33
Table 11 Create post Use Case	34
Table 12 Search Items Use Case.....	35
Table 13 Compare Items Use Case	35
Table 14 Get Recommendations Use Case.....	36
Table 15 Edit Post Use Case.....	36
Table 16 View Posts Use Case	37
Table 17 Purchase Item Use Case.....	37
Table 18 Delete posts Use Case.....	38
Table 19 Report Post Use Case.....	39
Table 20 Manage User Accounts Use Case	40
Table 21 Manage Users Posts Use Case	40
Table 22 System Monitoring Use Case	41
Table 23 Enforce TU Regulations Use Case	42
Table 24 Users Table	56
Table 25 Posts Table	57
Table 26 Reports Table	57
Table 27 Admins Table.....	58
Table 28 Requests Table	58
Table 29 Core Tools and Languages Used in TUMB.....	73
Table 30 Unit Testing	81

Table 31 Successful Flow Trigger (Post Flow)	82
Table 32 Successful Title Input (Post Flow).....	82
Table 33 Successful Flow Trigger (Buy Flow).....	82
Table 34 Successful Post ID Input (Buy Flow)	83
Table 35 Successful Flow Trigger (Compare Flow).....	83
Table 36 Unsuccessful Input – Same Post ID Twice (Compare Flow)	83
Table 37 Successful Flow Trigger (Report Flow)	84
Table 38 Successful Post ID Input (Report Flow)	84
Table 39 Successful Flow Trigger (Search Flow)	84
Table 40 Successful Category Input (Search Flow)	85
Table 41 Successful Flow Trigger (Edit Post Flow).....	85
Table 42 Successful Post ID Input (Edit Post Flow)	85
Table 43 Successful Flow Trigger (Delete Post Flow).....	86
Table 44 Successful Post ID Input (Delete Post Flow)	86
Table 45 Updated client-server architecture after the AI-agent extension	89
Table 46 MCP tools exposed by the TUMB MCP Server.....	91
Table 47 Comparative analysis of the rule-based and AI-powered agent approaches	100

List of Abbreviations

TUMB	Taibah University Marketplace Bot
TU	Taibah University
NLP	Natural Language Processing
UI	User Interface
API	Application Programming Interface
Fast-API	(Python Web Framework)
NLU	Natural Language Understanding
P2P	Peer-to-Peer
CI	Conversational interface
AI	Artificial Intelligence
WCAG	Web Content Accessibility Guidelines
UI	User interface
UX	User experience
UML	Unified Modeling Language
OOD	Object-Oriented Design
LLM	Large Language Model
MCP	Model Context Protocol

Chapter 1: Introduction

1.1 Introduction

Taibah University, an academic institution located in the holy city of Madina, Saudi Arabia, serves a vast academic community of approximately 45,000 students across its main campus and six regional branches, supported by a dedicated staff and faculty.

Throughout their academic journey, students are fundamentally required to utilize a wide array of educational materials and resources, required for their academic progression and achieving comprehensive learning outcomes.

Students at Taibah University often face high costs for essential academic supplies while used items from other students remain inaccessible due to the lack of a centralized trading platform. This fragmentation leads to unnecessary spending and waste.

This project directly addresses this gap by developing Taibah University Marketplace Bot (TUMB) app, a rule-based conversational app and a dedicated on-campus marketplace for the university community in Medina and its branches. It allows students to list items and use a chatbot to find, compare, and connect with sellers of university supplies, clothing, and lab equipment using natural language. By simplifying the peer-to-peer trading process, this app aims to create a more sustainable, affordable, and interconnected campus economy.

1.2 Problem Definition:

Students at Taibah University face significant financial burdens from the high cost of new academic supplies. The existing second-hand market is fragmented across inefficient channels like social media and notice boards, making it difficult for students to find, compare, and sell used items. This lack of a centralized, trusted platform creates information gaps and hinders cost-saving opportunities. This project is motivated by the need to solve these challenges with a single, streamlined, and intelligent marketplace for the university community.

1.3 Project Objectives

The main objective of app is to design and develop a rule-based chatbot that functions as the core interface for a marketplace platform. The specific objectives include:

- Develop a rule-based conversational app and a dedicated on-campus marketplace for the university community to find and compare two items based on price, details, and availability.
- Build a backend to manage user profiles, listings, and transactions.
- Create an intuitive interface for students to easily list and manage their items for sale.
- Establish an exclusive marketplace for the Taibah University community to ensure trusted transactions.

1.4 Project Scope

The scope of TUMB focuses on serving Taibah community, including students, staff and administrators with its branches.

1.5 Contributions of This Study:

This project's contributions directly benefit its main beneficiaries: Taibah University students, staff, and administrators by:

- Offering a platform to save money on essential supplies and earn from used items.
- Demonstrating a practical solution for e-commerce within the university context.
- Establishing a single, trusted marketplace for convenient peer-to-peer trading.

1.6 Project Timeline

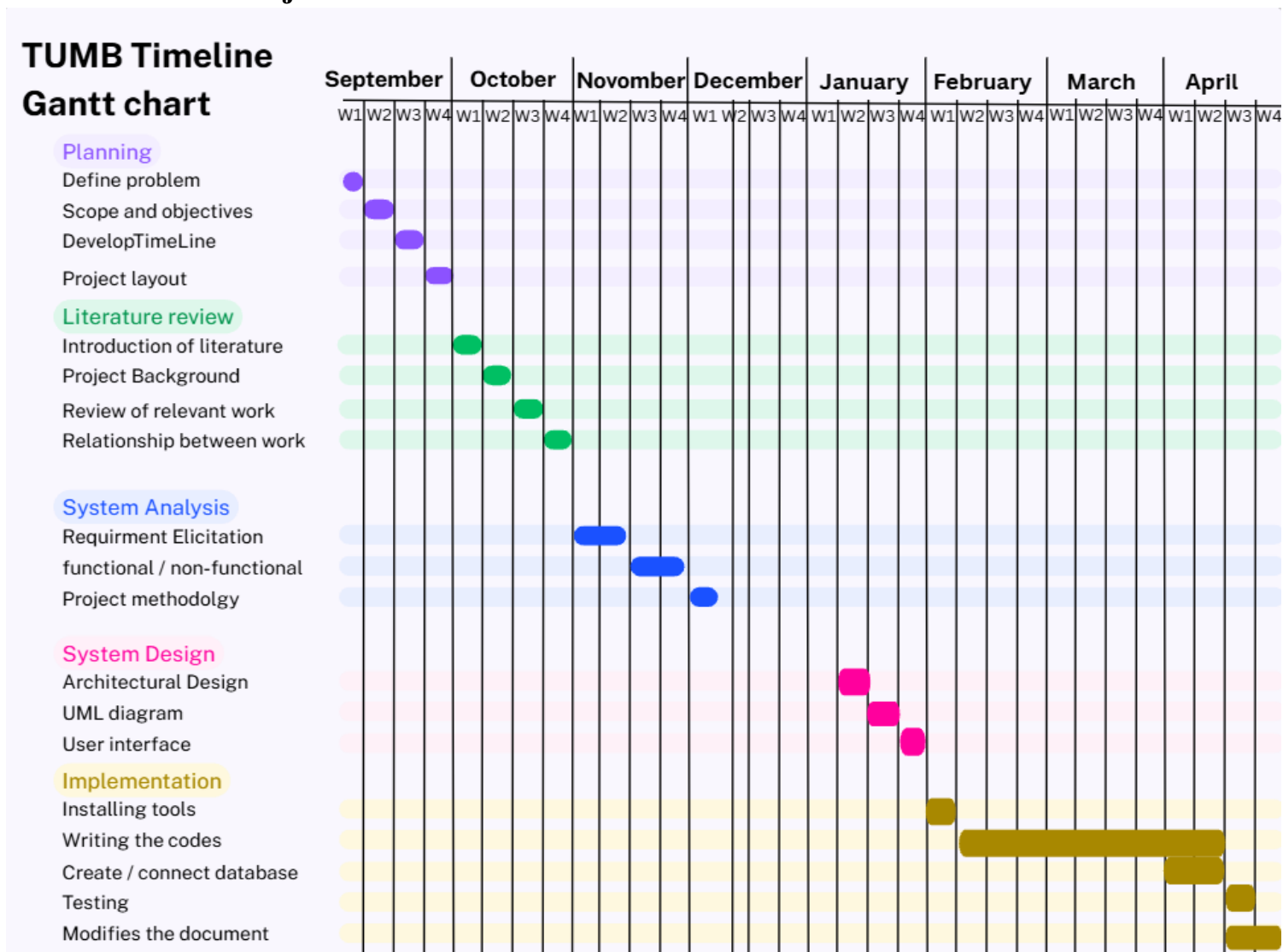


Figure 1 Gantt Chart

1.7 Document Organization

This document is structured as follows:

- **Chapter 1:** introduces the project, defines the problem, and outlines objectives, scope, and contributions.
- **Chapter 2:** resents a literature review of related systems and conversational models.
- **Chapter 3:** Describes system analysis and requirements specification.
- **Chapter 4:** presents the system design, including architecture, database design, and overall structure.

- **Chapter 5:** explains the system implementation, tools, technologies, and development process.
- **Chapter 6:** presents the case study of the AI agent extension (LLM), including its integration, functionality, and evaluation.
- **Chapter 7:** concludes the project, summarizing key results and suggesting future improvements.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews the theoretical background, related studies, and technological foundations that support the development of the Taibah University Marketplace Bot (TUMB) application. The focus is on exploring previous research and key concepts in online marketplaces, peer-to-peer trading, circular economy, conversational interfaces, and full-stack mobile application development. Understanding these domains provides the conceptual framework for developing.

2.2 Background

2.2.1 Marketplace and Second-Hand Goods

An online marketplace is a digital platform that connects buyers and sellers, facilitating the exchange of goods and services through a centralized system. Popular examples include Amazon, eBay, and Facebook Marketplace. In the academic context, second-hand marketplaces help students reduce costs by allowing them to buy and sell used books, electronics, and laboratory equipment within their campus community.

According to Kotler, marketplaces enhance accessibility and efficiency by removing the need for physical trading spaces. The second-hand market further promotes sustainability by extending the life cycle of resources and minimizing waste [1].

2.2.2 Usability and Accessibility

Usability and accessibility are essential aspects of any digital marketplace, ensuring that all users, regardless of technical skills or physical abilities, can easily navigate and interact with the application. Usability focuses on the intuitiveness, efficiency, and satisfaction experienced by users during interaction, following the principles outlined by Nielsen in the field of usability engineering [2]. Accessibility ensures inclusiveness by following standards such as the Web Content Accessibility Guidelines (WCAG), which provide design principles for color contrast, font readability, and compatibility with assistive technologies [3]. For the TUMB application, usability is enhanced through a simple chat-driven interface that reduces navigation complexity. Accessibility features

such as text-to-speech, adjustable font sizes, and language localization make the platform more inclusive to the diverse university community.

2.2.3 Payment Gateways

Payment gateways are secure digital systems that process online transactions between buyers and sellers. They encrypt sensitive payment data and ensure safe fund transfers using technologies such as SSL/TLS and tokenization, as discussed in secure payment architecture studies [4]. Common examples include Visa and Mada. Although the TUMB project prototype excludes real payment integration for security and scope reasons, understanding payment gateways is crucial for future system extensions. Implementing a trusted local payment API could enhance marketplace credibility and allow verified in-campus transactions with proper authentication and fraud protection mechanisms.

2.2.4 Shipping and Logistics

Shipping and logistics refer to the operational processes that handle the delivery of goods from sellers to buyers. In e-commerce platforms, efficient logistics systems ensure order tracking, delivery scheduling, and cost optimization, as highlighted in modern logistics research [5].

While the current TUMB system focuses on peer-to-peer transactions within the university and omits delivery modules, future versions could integrate logistics support for inter-branch item transfers or pickup coordination. This may include partnerships with local courier services or the use of smart lockers within the university to streamline item exchanges securely and efficiently.

2.2.5 Rule Engine

A rule engine is a software component that executes predefined business rules to automate decision-making processes. In a rule-based chatbot, the rule engine determines responses based on pattern matching, input triggers, and logical flow. It follows “if-then” conditions to guide user interactions and system actions consistently, as discussed in expert system design literature [6].

For the TUMB app, the rule engine forms the core of conversational logic — it manages conversation states, validates inputs, and triggers specific marketplace operations such as

item posting, searching, and comparison. Using a rule engine ensures predictability and control over chatbot behavior, which is ideal for a structured academic environment.

2.2.6 User Intent Recognition

User intent recognition involves identifying the goal or purpose behind a user's input. In conversational systems, it is critical for determining what the user wants to achieve, for example, buying, selling, or searching for an item. In rule-based chatbots, intent recognition typically relies on keyword detection and pre-defined pattern mapping, whereas AI-driven systems use Natural Language Understanding (NLU), as explained in modern Natural Language Processing NLP literature [7].

The TUMB application uses rule-based intent mapping, allowing the chatbot to interpret structured inputs with high accuracy and route the conversation accordingly.

2.2.7 Dialog Management

Dialog management governs the flow and structure of interactions between the user and the chatbot. It ensures that the system maintains logical progression across multiple turns of conversation, following established frameworks in dialogue system architecture [8]. For the TUMB app, dialog management follows a state-based design, where each state represents a conversational step (e.g., item search → confirmation → comparison). This modular structure simplifies debugging, maintains consistency, and ensures smooth transitions between intents. It also enables partial automation while retaining flexibility for future integration with AI-driven dialogue models.

2.2.8 Response Generation

Response generation is the process of producing appropriate system replies based on recognized intent and context. In rule-based systems, responses are predefined templates linked to specific conditions or user inputs, a method commonly used in structured conversational systems [9].

The TUMB app employs template-based response generation to maintain accuracy and clarity in communication. Each response is designed to be concise, informative, and action-oriented, guiding the user toward completing marketplace tasks such as viewing item details or comparing prices. This approach ensures consistent tone and minimizes ambiguity during interaction.

2.2.9 Context Handling

Context handling allows a chatbot to maintain awareness of previous user inputs and the overall conversation state. Effective context management prevents repetitive questioning and enables multi-turn interactions, a key principle in context-aware dialogue processing [10].

In the TUMB system, context handling is implemented through variable storage that tracks user identity, selected categories, and current conversation stage. This memory mechanism ensures that the chatbot can follow the flow of dialogue naturally — for example, recalling the last searched item or user preferences during a session. Proper context management enhances user experience and contributes to a seamless conversational flow.

2.2.10 Peer-to-Peer (P2P) Exchange

Peer-to-Peer (P2P) systems allow individuals to directly exchange goods or services without centralized intermediaries. In digital platforms like Haraaj [18], users interact directly, which reduces operational costs and fosters community-driven commerce.

Botsman and Rogers describe P2P exchange as a cornerstone of the sharing economy, where access is prioritized over ownership [11]. This model aligns well with university marketplaces, as it encourages collaboration and affordability among students.

2.2.11 Circular Economy and Resource Reuse

A circular economy is an economic model that aims to minimize waste and maximize resource reuse through repairing, recycling, and reselling. Unlike the traditional linear lifecycle (make–use–dispose), the circular model emphasizes long-term sustainability and efficient use of materials.

The proposed TUMB app supports the circular economy by enabling students to reuse academic materials and second-hand goods, reducing waste and promoting responsible consumption within the university community. According to the Ellen MacArthur Foundation, circular systems significantly reduce environmental impact by extending product life cycles [12].

2.2.12 Conversational Interface

A conversational interface (CI) allows users to interact with computer systems using natural language through text or voice. Popular examples include Siri, Alexa, and ChatGPT. These interfaces have become increasingly common in e-commerce and customer service due to their ease of use and accessibility.

In the context of a university marketplace, a conversational interface helps users quickly search items, communicate with sellers, and retrieve information without complex navigation. McTear emphasizes that well-designed conversational interfaces improve usability and reduce interaction barriers for non-technical users [13]

2.2.13 Artificial Intelligence and Rule-Based Systems

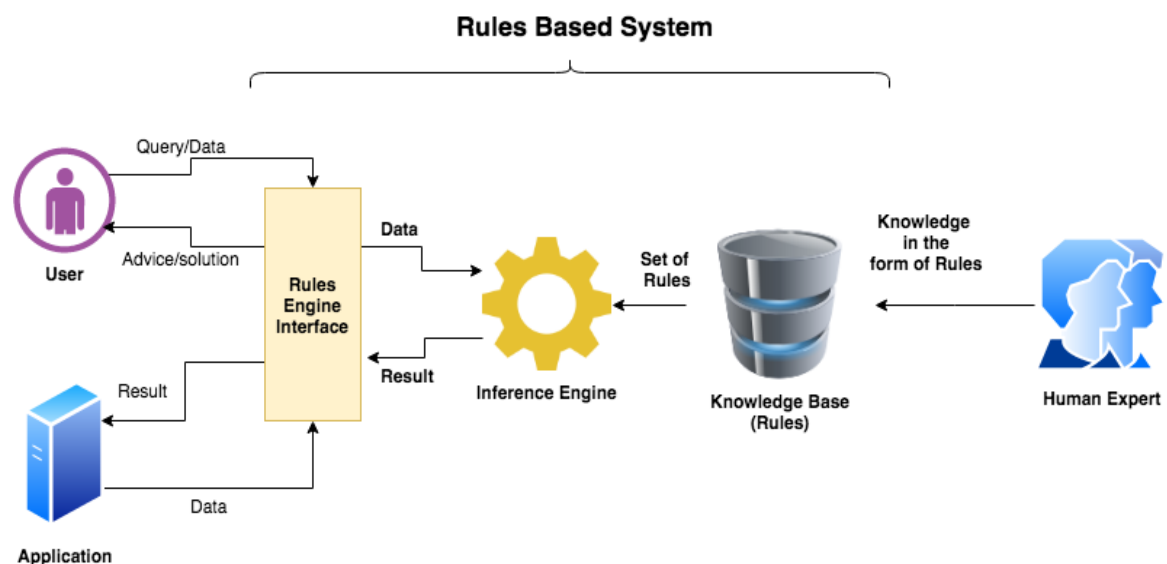


Figure 2 Artificial Intelligence and Rule-Based Systems

Artificial Intelligence (AI) refers to the development of computer systems capable of performing tasks that traditionally require human intelligence, such as learning, reasoning, problem-solving, and natural language understanding. According to McTear (2020), AI technologies have significantly transformed digital services, enabling systems to respond intelligently to user queries, automate decision-making, and improve user experience [13].

AI systems are generally categorized into two main types:

Rule-Based Systems – systems that operate using predefined rules and decision trees, where each user input triggers a specific programmed response.

Machine Learning Systems – systems that learn from data and improve their predictions over time.

For the TUMB application, a rule-based chatbot is selected instead of a machine-learning model. Rule-based systems are easier to control, provide predictable outputs, and are ideal for well-structured domains with limited and clear user intents. They rely on if-else logic flows, predefined patterns, and structured conversation steps to guide users through tasks such as registration, posting items, searching listings, and comparing products.

A rule-based conversation engine ensures that users receive accurate and consistent responses, which is critical for a university marketplace where information must be reliable and misunderstandings can disrupt the trading process. Additionally, rule-based systems require less computational power, no training datasets, and are easier to maintain — making them suitable for student-focused applications with limited development resources.

Chatbots are generally divided into two main categories: rule-based and AI-based. A rule-based chatbot relies on predefined patterns, keyword matching, and decision trees, producing predictable and controlled responses. In contrast, AI-based chatbots apply natural language processing (NLP) and machine learning to understand complex queries and learn from user interactions.

For the TUMB application, a rule-based chatbot is selected due to its simplicity, clear conversational scope, and predictable behavior. Rule-based systems are more suitable for structured domains such as academic marketplaces where queries are limited and well-defined. Shawar and Atwell highlight that rule-based chatbots provide high reliability when dealing with constrained domains and predefined tasks [14]

2.2.14 Software Development Stack

2.2.14.1 Full-Stack Development

Full-stack development refers to the implementation of both frontend (client-side) and backend (server-side) components. A full-stack developer manages all application layers to ensure consistent data flow, performance, and system integration. This approach is particularly suitable for academic projects where development teams operate with limited time and resources. Sommerville highlights that full-stack design promotes modularity and simplifies system maintenance across multiple layers [15].

2.2.14.2 Frontend Development

The frontend handles the user interface (UI) and user experience (UX), enabling users to interact with the system through intuitive layouts and responsive components. Frontend applications connect to the backend using RESTful APIs. Common frontend technologies include React and Flutter, which provide reusable components and dynamic interfaces. MDN documentation emphasizes the importance of clean API communication to maintain a seamless user experience across devices [16].

2.2.14.3 Backend Development

The backend consists of server-side logic responsible for authentication, data processing, and communication with the database. Frameworks such as FastAPI and Node.js are widely used for building scalable APIs. According to Sommerville, the backend must ensure security, data consistency, and reliable communication with external components [15].

2.2.14.4 React Frontend Library

React, developed by Meta, is a JavaScript library designed for building dynamic user interfaces based on a component architecture. Components improve code reuse, simplify maintenance, and support modular design. React's virtual DOM also enhances application performance. Google Developers highlight that component-based architectures improve UI scalability and maintainability in modern applications [17].

2.2.14.5 FastAPI Backend Framework

FastAPI, a modern Python web framework designed for building high-performance APIs, provides automatic request validation, built-in interactive API documentation, and follows widely adopted standards such as OpenAPI. These capabilities can enhance the backend of the TUMB application by enabling reliable and scalable data processing, secure user management, and seamless integration with the frontend through RESTful APIs. Security features are supported to help protect user data and system integrity.

2.2.14.6 RESTful API Endpoints

REST is an architectural style for web services. RESTful APIs provide structured endpoints for communication between the frontend and backend. For example, GET /items retrieves resources, while POST /login handles user authentication. Requests and

responses are usually in JSON, ensuring consistent and scalable interaction within the TUMB application.

2.2.15 A UI/UX Design

User Interface (UI) and User Experience (UX) design play a critical role in determining how users perceive and interact with digital systems. UI design focuses on the visual layout —colors, typography, buttons, icons, and other interface components that make an application aesthetically appealing and easy to navigate. UX design, on the other hand, emphasizes the overall journey of the user, ensuring that every interaction is smooth, intuitive, and goal oriented.

In the context of the (TUMB) application, effective UI/UX design ensures that students and staff can quickly perform essential tasks such as posting items, searching for supplies, or chatting with the rule-based assistant. A clean, minimal, and consistent interface helps reduce cognitive load, allowing users to focus on completing their objectives rather than learning how to navigate the system.

The design process for UI/UX typically involves several iterative steps:

User Research and Persona Development – Understanding the target audience’s needs, behaviors, and technical proficiency.

Wireframing and Prototyping – Creating low- and high-fidelity mockups to visualize user flows and layout structures.

Usability Testing – Conducting tests with representative users to identify bottlenecks or confusing elements before final deployment.

Responsive Design – Ensuring the interface adapts seamlessly across multiple screen sizes and devices.

Accessibility Optimization – Applying design standards such as WCAG 2.1 to ensure inclusiveness for all users, including those with visual or motor impairments.

For TUMB, the front-end development frameworks **Flutter** and **React Native** provide built-in components and responsive layout systems that support efficient UI/UX implementation. These frameworks also allow for integration with **Material Design** and **Apple Human Interface Guidelines**, ensuring the application adheres to modern usability and accessibility standards.

A well-designed UI/UX not only improves satisfaction but also builds **trust** and **retention**, which are crucial for marketplace systems where user engagement directly influences success.

2.2.16 API and CRUD Operations

An **Application Programming Interface (API)** enables communication between different software components. REST APIs allow the frontend to interact with the backend efficiently using standard HTTP methods.

CRUD operations — *Create, Read, Update, and Delete* — represent the fundamental database actions performed through APIs. These operations form the basis for managing users, listings, and messages within the marketplace system.

2.2.17 Database Management and Mobile Development

2.2.17.1 Database Management

Databases are essential for storing and managing structured and semi-structured data. In marketplace systems, they handle user profiles, product listings, and transaction records. PostgreSQL, a SQL database, is preferred for its reliability, scalability, and strong support for structured data. It also supports JSON data types, allowing flexibility to meet the evolving needs of marketplace platforms.

2.2.17.2 Chosen Database in the TUMB Application

In this project, **PostgreSQL** will be used as the primary database for managing user accounts, product listings, and marketplace interactions. PostgreSQL is used because it offers strong support for structured data along with reliable performance. Its scalability makes it suitable for real-time mobile applications, and its support for JSON data types allows flexible data storage when needed. The database will be integrated with the backend through RESTful APIs built using FastAPI, ensuring secure and efficient communication between the server and the mobile app.

2.2.18 Mobile Development

Mobile development refers to the process of designing and building software applications that run on mobile devices such as smartphones and tablets. In recent years, mobile applications have become the primary way users interact with digital services due to their

convenience, accessibility, and intuitive interfaces. Mobile applications can be developed using native, hybrid, or cross-platform approaches. Native development, such as Kotlin for Android or Swift for iOS, provides high performance but requires maintaining separate codebases for each platform, increasing development time and effort.

To address this challenge, cross-platform frameworks such as Flutter and React Native have gained popularity. These frameworks enable developers to write a single codebase that can run on both Android and iOS devices, significantly reducing development time while maintaining performance and usability. Flutter, developed by Google, uses the Dart programming language and provides a fast development cycle and a rich set of customizable UI components. React Native, developed by Meta, allows developers to build mobile apps using JavaScript and React components while accessing native device features.

In the TUMB application, a cross-platform mobile development framework is utilized to ensure that the system is easily accessible to all students regardless of the device they use. The application is designed to offer an intuitive user interface, smooth navigation, and real-time data interaction with the backend. This ensures that users can browse items, communicate with sellers, and manage their marketplace activities anytime and anywhere, improving the overall usability and engagement of the platform.

2.2.19 UML Modeling Tools

UML (Unified Modeling Language) is essential for visualizing the structure and behavior of the Rule-Based Marketplace, guiding the development of component interactions.

The following tools are selected for their suitability and features:

Tool	Primary Advantage	Key Use in Project
Lucidchart	Web-based and collaborative	Quick design and sharing of high-level diagrams and system architecture.
StarUML	Powerful desktop modeling	Generating detailed Class Diagrams (for PostgreSQL/backend) and complex Sequence Diagrams .

Tool	Primary Advantage	Key Use in Project
PlanTUMB L	Code-as-diagram approach	Storing diagrams as text files within the project's version control (Git) for easy maintenance.

Table 1 UML Modeling Tools

2.3 Related Work

This section presents reviews of related work to our rule-based marketplace chatbot system. It highlights their features and limitations to demonstrate how our proposed system enhances user interaction and marketplace efficiency through rule-based automation.

Following a comprehensive review of related work, no existing system was identified that directly aligns with the core concept of the TUMB application. Available platforms either target general public audiences for commercial trading, focus on non-academic product categories, or lack the university-exclusive, rule-based conversational marketplace design that distinguishes TUMB from existing solutions.

2.3.1 Review of Relevant Work

Haraj



Haraj is one of the largest online marketplace platforms in Saudi Arabia, enabling users to post, browse, and purchase items in various categories such as vehicles, electronics, and real estate. The system primarily depends on manual navigation and keyword searches. It works by allowing users to create accounts, post advertisements for their products, and communicate directly with buyers or sellers through chat or phone calls [18].



Figure 3 Haraj

Aspect	Platform Features	Platform Limitations	Proposed Enhancement
User Interaction	Web-based platform with multiple pages for browsing, posting, and searching items.	Requires familiarity with website navigation and menu structures to complete tasks.	Conversational flow simplifies user interaction through guided, rule-based prompts that reduce cognitive load.
Posting Efficiency	Form-based listing creation with all fields presented simultaneously on single page.	Users must independently identify and complete required fields without guidance.	Sequential question-and-answer flow guides users through each required field step by step, with final confirmation at the end.
Item Comparison	No comparison functionality available; users must view items in separate tabs or windows.	Students cannot easily evaluate multiple options when seeking cost-effective academic supplies.	Integrated comparison feature enables users to choose two items and view their differences directly in chat.

Transaction Model	Direct communication between buyers and sellers through chat or phone calls.	No built-in notification or confirmation system confirming user interest.	Automated email notification feature informs the seller when a buyer shows interest, improving response speed without handling payments
Market Scope	Local public platform serving nationwide audience across diverse product categories.	Broad audience creates uncertainty about seller reliability and transaction safety for students.	Focused on Taibah university community creating trusted environment for student-to-student exchanges of academic items.

Table 2 Comparison with Haraj Platform



Facebook Marketplace

Facebook Marketplace allows users to buy and sell items within their social network. The platform provides automated product suggestions and category-based browsing but lacks a conversational or guided interface.

It works by letting users list items for sale, browse nearby listings, and communicate with buyers or sellers through Facebook Messenger. The marketplace automatically suggests products based on user activity and location [19].

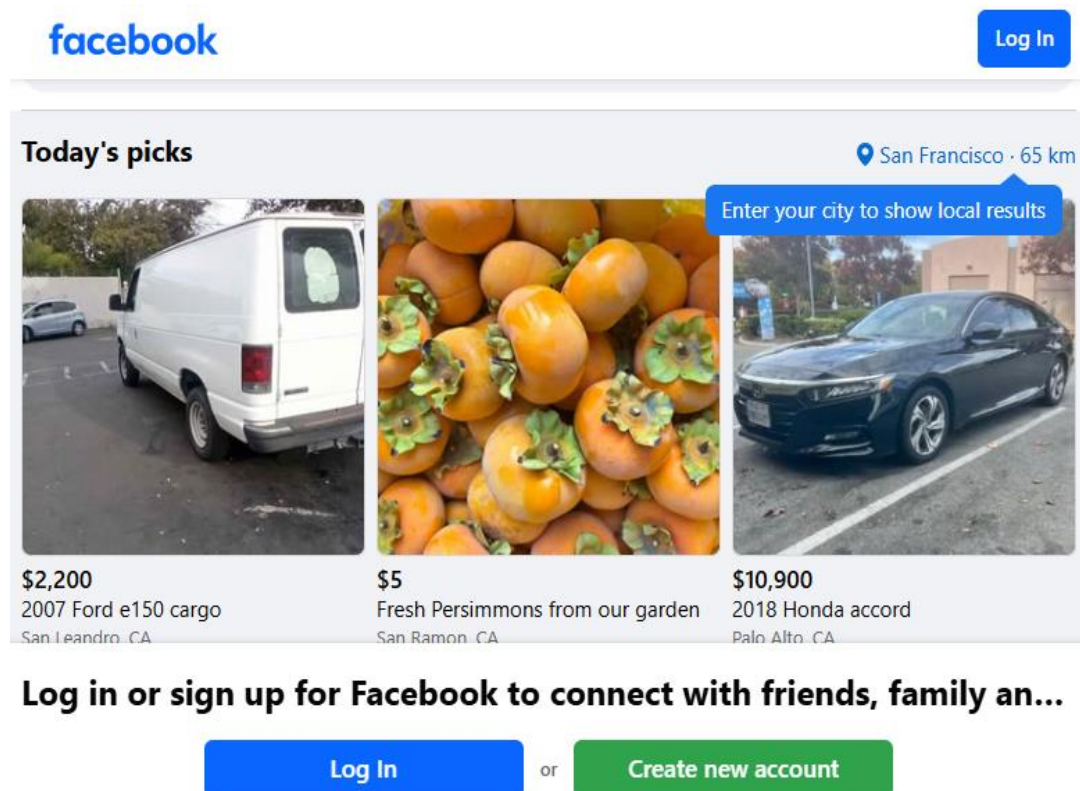


Figure 4 Facebook marketplace

Aspect	Platform Features	Platform Limitations	Proposed Enhancement
User Interaction	Web-based interface integrated within Facebook's social platform, allowing users to browse and communicate through Messenger.	Interface design depends on Facebook's general layout, which mixes social elements with marketplace navigation and requires a personal Facebook account.	Login limited to university email accounts, offering a simplified standalone conversational interface that guides users through marketplace interactions step by step.
Posting Efficiency	Allows quick posting with automatic category suggestions based on previous activity.	Lacks structured guidance, relying on user familiarity with Facebook interface.	Sequential question-and-answer flow guides users through each required field step by step, with final confirmation at the end.
Item Comparison	No integrated comparison function; users manually check	Students spend extra time reviewing prices and details to find	Integrated comparison feature enables users to choose two items and view

	multiple listings or tabs.	suitable academic supplies.	their differences directly in chat.
Transaction Model	Fully integrated online payment and shipping system for global transactions.	Complex infrastructure unnecessary for simple local exchanges.	Simplified email-based notification informs the seller of buyer interest for local university transactions only.
Market Scope	Public marketplace accessible nationwide across diverse product types.	Mixed and irrelevant listings make it difficult for students to find academic-related items.	Localized marketplace focused exclusively on Taibah University, filtering content to academic supply trading only.

Table 3 Comparison with Facebook Platform



TreasureX

TreasureX is a university-exclusive online marketplace platform designed specifically for UC Berkeley students to buy and sell secondhand items within their campus community. The platform requires users to register with a Berkeley.edu email address, creating a trusted environment limited to verified students. The system features a simplified three-step posting process where users add a title, upload a photo, and submit their listing. TreasureX offers pre-made category lists including Free, Graduation, Furniture, School, Electronics, Apparel, and Home, displaying search results in order. The platform includes a built-in direct messaging system that enables buyers and sellers to coordinate transaction details such as payment and pickup locations. TreasureX aims to promote affordable campus living and sustainability by facilitating the reuse of academic supplies and reducing waste from discarded items during move-in and move-out seasons [20].

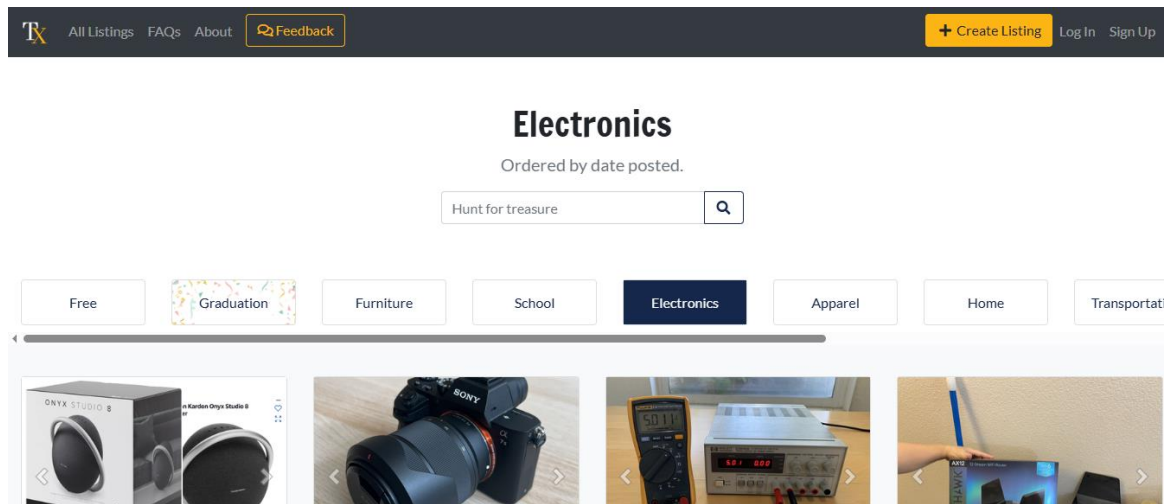


Figure 5 TreasureX

Aspect	Platform Features	Platform Limitations	Proposed Enhancement
User Interaction	Web-based platform with simplified three-step posting process requiring page navigation	Users must navigate website structure and locate forms to complete tasks	Chat-based conversational interface guides users through sequential prompts without navigation requirements
Posting Efficiency	Streamlined form submission focused on essential fields for student marketplace transactions	Posts may lack sufficient details as platform does not enforce description or category completeness	Rule-based validation requires complete information including description and category before posting acceptance
Item Comparison	Pre-made category lists displaying items without comparison functionality	Students must manually track multiple listings to compare prices and identify cost-saving opportunities	Integrated comparison feature allows direct side-by-side evaluation of selected items within chat

Communication Efficiency	Built-in messaging system requiring direct communication between buyer and seller	Message exchanges needed to express interest and coordinate transaction details	Automated email notification sends buyer contact information to seller immediately upon purchase interest
--------------------------	---	---	---

Table 4 Comparison with TreasureX Platform



KFUPM Store

KFUPM Store is the official online store of King Fahd University of Petroleum and Minerals, located in Dhahran, Saudi Arabia. The platform offers a wide range of university-branded products such as clothing, bags, stationery, drinkware, accessories, and graduation items. Products are organized into clearly defined categories to make browsing simple and efficient. The website features sections like Most Popular and New Arrivals to highlight best-selling and newly added items. KFUPM Store supports secure online payment and provides shipping across Saudi Arabia, making it accessible to students, staff, graduates, and the public.

The official website of KFUPM Store is available at [21].

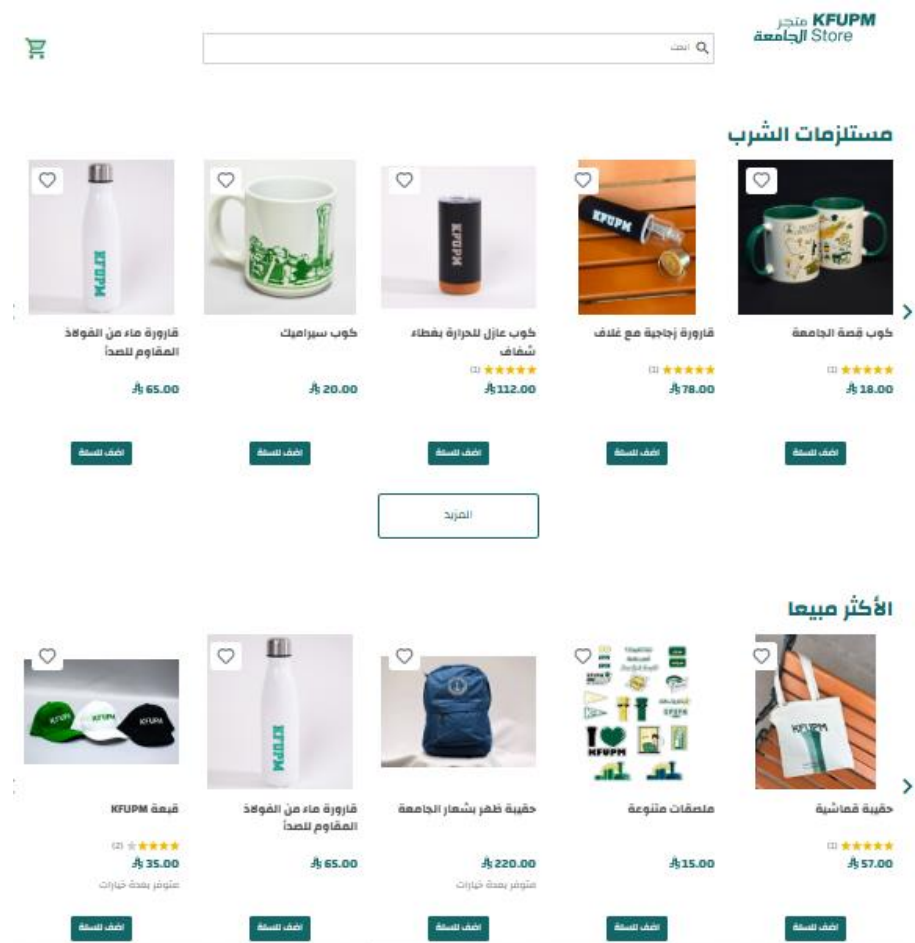


Figure 6 KFUPM Store

Aspect	Platform Features	Platform Limitations	Proposed Enhancement
User Interaction	Web-based e-commerce platform with catalog browsing organized by product categories	Requires familiarity with website navigation and menu structures to complete tasks.	Chat-based conversational interface guides students through marketplace actions without requiring navigation effort
Posting Efficiency	Pre-listed inventory managed by store with product descriptions and images provided	Users cannot post academic items or contribute listings to the platform	Sequential question-and-answer flow enables users to complete each required field step by step with final confirmation at the end

Item Comparison	No comparison functionality available; users must view items in separate tabs or windows.	No comparison tool for evaluating items or identifying cost-efficient alternatives	Integrated comparison feature enables students to choose two items and view their differences directly in chat
Market Scope	Open to public users across Saudi Arabia including non-KFUPM members	Does not provide features tailored to academic item exchange within student community	Focused on Taibah university community creating trusted environment for student-to-student exchanges of academic items

Table 5 Comparison with KFUPM Store Platform



Telegram Groups

Telegram groups are one of the most common and usually unofficial methods students in Saudi universities use to trade secondhand academic supplies. Many of these groups are created for general university-related discussions, with academic-item exchanges occurring as just one of several uses. Students typically post items in chat threads and wait for responses, with no automation or structured process involved. Because these groups are unregulated and lack student verification, scammers can easily join and pose as students, creating trust and safety issues. Messages also get buried quickly as new posts appear, forcing sellers to repost frequently, while buyers struggle to know whether an item is still available. Communication delays are common, and many exchanges shift to WhatsApp for identity confirmation. Despite these limitations, Telegram groups remain widely used among university students for informal trading.

Examples of commonly used public Telegram groups include:

https://t.me/G_TaibahuD

<https://t.me/aaa2222p>

https://t.me/kau2020_2

<https://t.me/+ahtuTIIFPBswYmY0>

<https://t.me/KFUPMz>

<https://t.me/UQUniversity>

(These groups are unofficial, not reliable, and their links may change at any time.)



Figure 7 KFUPM telegram market



Figure 8 TU telegram group for second-hand market

Aspect	Platform Features	Platform Limitations	Proposed Enhancement
User Interaction	Unstructured group chat where students manually post and browse messages about items	Free-form messaging creates cognitive overload; students must scroll through numerous unorganized messages to find relevant items	Chat-based conversational interface with rule-based prompts guides students through structured posting and searching processes
Posting Efficiency	Students write messages describing items in any format with no required fields or structure	Posts often lack essential details; no validation mechanism ensures complete information leading to unclear listings	Sequential question-and-answer flow ensures each required field is completed step by step with final confirmation
Item Comparison	Students must manually track multiple messages across the chat to compare different items	No comparison functionality; students waste time noting prices and details from scattered messages	Integrated comparison feature enables students to select two items and view their differences directly in chat
Trust and Verification	Open groups accessible to anyone with the group link without membership restrictions	No identity verification; participants remain anonymous increasing fraud risk and distrust among students	Registration requires university email providing identifiable student identity and creating trusted trading environment

Communication Efficiency	Built-in Telegram messaging enables direct buyer-seller communication within the platform	Back-and-forth messaging time-consuming; sellers often redirect to WhatsApp to reveal phone numbers and build greater trust	Automated email notification informs seller when buyer shows interest with contact details, reducing response time and communication barriers
--------------------------	---	---	---

Table 6 Comparison with Telegram Groups

2.3.2 Relationship Between Relevant Work and Our Own Work

The reviewed systems share the goal of connecting buyers and sellers efficiently; however, most rely on manual navigation and static interfaces. Our proposed rule-based chatbot marketplace introduces structured conversational flows that automate each user action — from registration and login to posting, searching, comparing, and receiving recommendations — through logical, step-by-step interaction.

Features	TUMB	Haraj	Facebook Marketplace	TreasureX	KFUPM Store	Telegram Groups
Rule-based chat interaction	✓	✗	✗	✗	✗	✗
Automated posting guidance	✓	✗	✗	✗	✗	✗
Integrated item comparison	✓	✗	✗	✗	✗	✗
Personalized recommendations	✓	✗	✓	✗	✗	✗
Step-by-step conversation flow	✓	✗	✗	✗	✗	✗

User registration through chat	✓	✗	✗	✗	✗	✗
--------------------------------	---	---	---	---	---	---

Table 7 Relationship Between Works

2.4 Summary

This chapter explores the theoretical and technological foundations underlying the TUMB application. It reviewed the concepts of online marketplaces, peer-to-peer trading, and circular economy as the economic basis of the system and examined conversational interfaces and rule-based chatbots as the interaction model. Furthermore, it discussed the full-stack architecture, API communication, CRUD operations, UML tools, database management, mobile application, and related works. The next chapter will analyze the system requirements and provide a detailed overview of its analysis phase.

Chapter 3: System Analysis

3.1 Introduction

System analysis is a vital phase in developing the Taibah University Marketplace Bot (TUMB) application, as it defines the system's needs and ensures that the proposed rule-based chatbot meets user and institutional requirements. This stage identifies current problems, outlines system functions and constraints, and establishes the foundation for design and implementation.

This chapter analyzes existing marketplace systems, identifies functional and non-functional requirements, defines user roles, and presents the methodology used to develop the TUMB platform.

3.2 Requirements Elicitation

The requirements are the descriptions of the system services and constraints.

3.2.1 Functional Requirements

The system should allow users to create an account by entering a valid email, name, and password.

The system should allow users to log in using their correct email and password.

The chatbot should allow users to search for items by typing a name or choosing a category.

After searching, the system should allow users to **compare items** by selecting two or more item numbers.

After searching, the system should allow users to **request to purchase an item**, and the system should notify the seller.

The system should allow users to **create new posts** by entering item details such as name, price, and description.

The system should allow users to **edit their posts** by choosing a post and updating its information.

The system should allow users to **delete their posts** permanently after confirmation.

The chatbot should allow users to view all posts they have created.

The System should provide users with **recommended items** based on their interests.

The system should allow users to **report posts** that are inappropriate or misleading.

The system should allow administrators to **manage users' accounts**, including reviewing, suspending, or removing accounts.

The system should allow administrators to **manage users' posts**, including editing or deleting posts when necessary.

The system should allow administrators to perform **system monitoring**, such as checking system status and reviewing reports.

3.2.2 Non-Functional Requirements

Non-Functional Requirement	Description	Measurement
Performance	The chatbot should respond quickly when users type messages.	The chatbot should reply to user messages within 2–3 seconds during normal use. During testing, the response time will be measured by sending multiple messages and checking how long it takes for the system to respond. If more than 10% of the replies take longer than 3 seconds, the performance is considered below the requirement.
Availability	The system should be available most of the time for users to access.	The system should be available at least 95% of the time each month . Availability will be checked by observing whether the system can be accessed at different times of the day over a one-month period. If the system is offline for more than 5% of the time , it does not meet the availability requirement.
Security	User information should be kept safe and private.	User passwords must be stored in a protected format so they cannot be read by anyone.

		During testing, different user accounts will be checked to confirm that one user cannot view or access another user's information. If any user can see or access someone else's data, the requirement is considered not met.
Usability	The system should be easy to use and understand.	A group of test users will try the chatbot, and at least 80% of them should report that it is "easy to use." Users should be able to complete basic tasks (such as searching or posting) without needing instructions. If users struggle to complete common actions, the system does not meet the usability requirement.
Reliability	The system should work correctly without errors or crashes.	The system will be used repeatedly for common tasks (searching, posting, editing, etc.) for 50 continuous tests, and it should complete these tasks without crashing. The system should remain stable even when multiple actions are performed quickly or under slightly heavy usage. If more than 1 out of 20 tests results in an error or crash, the reliability requirement is not met.

Table 8 Non-Functional Requirements

3.2.3 User Requirements or Domain Requirements

To use the chatbot marketplace, the user must have a device with internet access.

Users must create an account with a valid university email and password before using the system.

Only registered university students should be allowed to create accounts and access the system.

With the correct login information, the user should be able to access the system at any time.

Users should be able to search for items by typing the item name or selecting a category.

After searching, users should be able to compare items and view their details easily.

Users should be able to report any post they find inappropriate, misleading.

Users must be able to edit their own posts by choosing the post and updating its information.

Users must be able to delete their own posts permanently if they no longer need them.

After finding an item, users should be able to request to purchase it through the chatbot.

The system should provide users with recommended items based on their interests.

The system should help students find items related to their departments (e.g., engineering tools, medical lab kits, art supplies).

The system should display all posts created by the user when requested.

All posts should be related to items commonly used by university students (books, electronics, lab equipment, clothing, etc.).

To maintain a safe marketplace, administrators must be able to manage user accounts and posts.

The marketplace should prevent the posting of items that could harm students or violate university regulations, such as chemicals, dangerous tools, or restricted materials.

The system should ensure that all posts follow university policies, including safety rules and student conduct guidelines.

3.3 Requirements Specification

Use cases provide a structured way to describe how different users interact with the system to achieve specific goals. They help identify the main functions of the system, the roles of each actor, and the sequence of steps involved in completing various tasks. By outlining the expected user interactions, use cases ensure that the system's requirements are well understood and accurately represented during the design and development stages. In this project, use cases illustrate how students, administrators, and the database work together to support key marketplace activities such as searching for items, posting listings, managing content, and ensuring a safe and organized platform for university students.

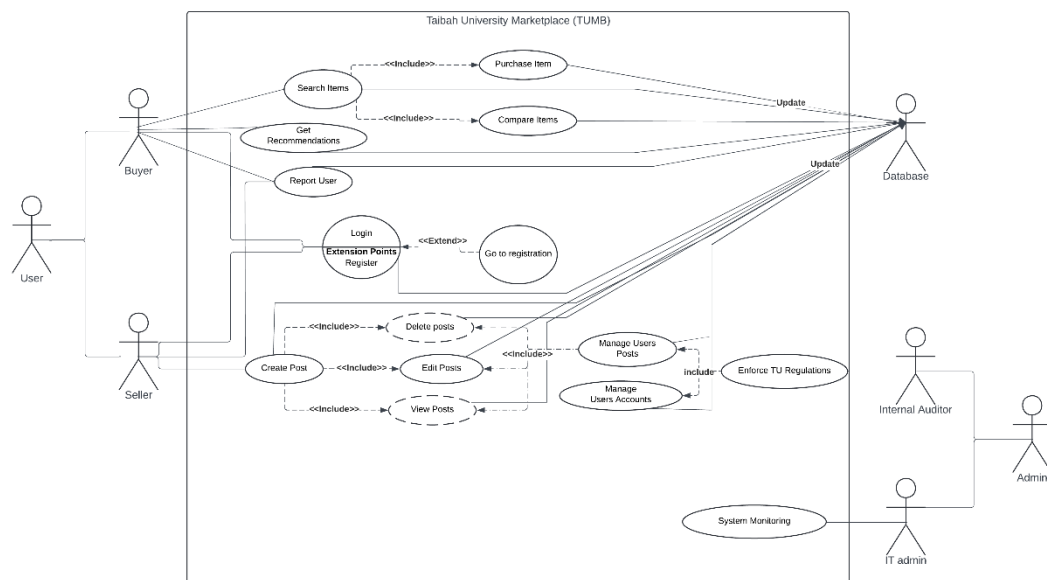


Figure 9 Requirements Specification

Use case name	Register
Actors	User, Database
Description	Allows a new user to create an account by entering their name, email, and password.
Entry Condition	The user opens the chatbot and selects “Register.”

Basic flow	1. The chatbot asks for name, email, and password. 2. The user enters the required details. 3. The system saves the new account.
Exit condition	The user's account is successfully created and stored in the database.
Quality Requirement	Registration should complete within a few seconds, and confirmation should be clearly shown.

Table 9 Register Use Case

Use case name	Login
Actors	User, Database
Description	Allows registered users to access the system by verifying their credentials.
Entry Condition	The user has already registered an account.
Basic flow	1. The chatbot asks for email and password. 2. The user enters valid credentials. 3. The chatbot verifies them and grants access.
Exit condition	The user is logged in and can see the main menu.
Quality Requirement	Login should be fast, and incorrect credentials should show a clear error message.

Table 10 Login Use Case

Use case name	Create post
---------------	-------------

Actors	User, Database
Description	Allows a user to create a new product listing.
Entry Condition	The user is logged in and Type “Post” from the menu.
Basic flow	1. The chatbot asks for category and item name. 2. The user provides keywords. 3. The chatbot shows matching results with item details.
Exit condition	The new item post is stored and visible to other users.
Quality Requirement	The process should be simple, and all required fields must be filled correctly.

Table 11 Create post Use Case

Use case name	Search Items
Actors	User, Database
Description	Allows users to search for items by name or category.
Entry Condition	The user is logged in and Type “Search.”
Basic flow	1. The chatbot asks for category and item name. 2. The user provides keywords. 3. The chatbot shows matching results with item details.
Exit condition	The user sees a list of results or a message if nothing is found.

Quality Requirement	Search results should appear quickly and be relevant to the keywords.
---------------------	---

Table 12 Search Items Use Case

Actors	User, Database
Description	Compare two or more items based on price and attributes.
Entry Condition	The user has already performed a search.
Basic flow	1. The user selects two or more items from the search results. 2. The system retrieves detailed information about each item. 3. The system generates and displays a clear comparison table.
Exit condition	The comparison is shown clearly, helping the user make a decision.
Quality Requirement	The comparison should be accurate and easy to read.

Table 13 Compare Items Use Case

Use case name	Get Recommendations
Actors	User, Database
Description	Provides users with recommended items based on their interests.
Entry Condition	The user is logged in and type “Recommendations.”
Basic flow	1. The chatbot checks the user’s interest categories. 2. The chatbot shows related items.

Exit condition	A list of recommended items is displayed to the user.
Quality Requirement	Recommendations should be relevant to the user's past activity or categories.

Table 14 Get Recommendations Use Case

Use case name	Edit Post
Actors	User, Database, Admin
Description	Allows modification of an existing post.
Entry Condition	The user is logged in and has existing posts.
Basic flow	1. The chatbot asks which post to edit. 2. The user selects the post ID and the field to change. 3. The chatbot updates the post after confirmation.
Exit condition	The post is updated successfully and saved in the database.
Quality Requirement	The update should be confirmed clearly and completed without errors.

Table 15 Edit Post Use Case

Use case name	View Posts
Actors	User, Database, Admin
Description	Displays all posts created by the logged-in user or all posts for the admin.
Entry Condition	1. The user selects "View Posts," OR

	2. The admin views posts through the management panel.
Basic flow	1. The chatbot retrieves all posts created by the user. 2. The chatbot displays each post with details.
Exit condition	The user can see all their posts in a list format.
Quality Requirement	All posts should be displayed correctly and quickly.

Table 16 View Posts Use Case

Use case name	Purchase Item
Actors	User, Database
Description	Confirm interest in an item and notify the seller.
Entry Condition	Allows a buyer to express interest in purchasing a specific item.
Basic flow	1. The user selects an item and confirms interest. 2. The chatbot asks for confirmation. 3. When confirmed, the system notifies the seller by email.
Exit condition	The seller receives a notification, and the user gets confirmation.
Quality Requirement	The process should be clear and secure, ensuring accurate contact information.

Table 17 Purchase Item Use Case

Use case name	Delete posts
---------------	--------------

Actors	User, Database, Admin
Description	Allows users or administrators to delete an existing post.
Entry Condition	1. The user has at least one post, OR 2. The admin is managing posts.
Basic flow	1- The chatbot asks the user which post they want to delete. 2- The user enters the Post ID. 3- The chatbot asks for confirmation. 4- The user confirms “Yes”. 5- The system deletes the post.
Exit condition	The selected post is removed from the database.
Quality Requirement	The system should clearly confirm the deletion to prevent mistakes.

Table 18 Delete posts Use Case

Use case name	Report Post
Actors	User, Database
Description	Allows the user to report inappropriate, offensive, or misleading posts.
Entry Condition	The user must be logged in and must have found a post they want to report.
Basic flow	1- The user selects the option to report a post. 2- The chatbot asks the user to enter the Post ID they want to report. 3- The user provides the Post ID

	4- The chatbot asks for the reason for reporting the post. 5- The user provides a reason (e.g., inappropriate, scam, offensive). 6- The chatbot confirms the report and sends it to the administrator.
Exit condition	The report is successfully submitted and stored for the administrator to review.
Quality Requirement	The system should clearly confirm the report and ensure that the report is delivered to an administrator without delay.

Table 19 Report Post Use Case

Use case name	Manage User Accounts
Actors	Admin , Database
Description	Allows the administrator to view, suspend, reactivate, or delete user accounts.
Entry Condition	Admin is logged into the admin panel.
Basic flow	1- The admin selects "Manage Users". 2- The system displays all registered users. 3- Admin may view, suspend, or delete user accounts.
Exit condition	User account updates are stored and reflected immediately

Quality Requirement	Admin actions should be secure and require confirmation before changes.
---------------------	---

Table 20 Manage User Accounts Use Case

Use case name	Manage Users Posts
Actors	Admin, Database
Description	Allows administrators to manage all posts in the marketplace.
Entry Condition	Admin is logged into the admin dashboard.
Basic flow	1- The system shows all posts in the marketplace. 2- Admin selects a post to review. 3- Admin may edit, hide, or delete inappropriate posts.
Exit condition	Changes to posts are saved and updated in the system
Quality Requirement	The system must ensure only admins can modify other users' posts

Table 21 Manage Users Posts Use Case

Use case name	System Monitoring
Actors	Admin, Database
Description	Allows the admin to monitor system performance
Entry Condition	Admin is logged into the admin dashboard.
Basic flow	1- Admin checks system health (database connection, server status).

	2- Admin reviews logs or error reports.
Exit condition	The system remains stable and updated after admin actions
Quality Requirement	Monitoring tools should be easy for the admin to understand; system downtime should be minimized.

Table 22 System Monitoring Use Case

Use case name	Enforce TU Regulations
Actors	Admin
Description	Admin ensures that all posts follow Taibah University rules. The administrator reviews flagged content, removes inappropriate posts, and takes action against users who violate regulations to keep the marketplace safe and suitable for students.
Entry Condition	1- Admin is logged into the admin dashboard. 2- The system must have access to the list of TU rules and prohibited items. 3- At least one post or user activity requires review (system-flagged or user-reported).
Basic flow	1-The administrator navigates to the “Regulation Enforcement” dashboard. 2-The system displays all flagged or suspicious posts. 3-The administrator selects a post to review.

	4-The system shows details such as item name, description, category, and user information. 5-The administrator compares the content against TU regulations. 6-If the post violates rules, the administrator removes it and optionally warns the user. 7-If violations are repeated, the administrator suspends or restricts the user's account.
Exit condition	1- Violating posts are removed or corrected. 2- Responsible users are warned, restricted, or suspended as needed. 3-The system updates the status of reviewed posts. 4- A safer and TU-compliant marketplace environment is ensured.
Quality Requirement	1- At least 95% of clearly violating posts should be caught either by the system or administrators. 2- Administrator action logs must be stored securely for future audits.

Table 23 Enforce TU Regulations Use Case

3.4 Developmental Methodology

System development for the Taibah University Marketplace Bot (TUMB) followed a systematic and iterative approach to ensure that the application met the needs of students and provided a reliable platform for on-campus trading. The process included requirements analysis, system design, implementation, testing, and evaluation, and each

stage produced outputs such as requirement lists, design diagrams, and test results that guided the next phase and ensured smooth progress and consistent quality.

3.4.1 Waterfall Model-Design

The team followed a structured Waterfall development approach, where each phase of the system—requirements analysis, design, implementation, testing, and deployment—was completed in a sequential manner. This allowed the rule-based chatbot, the FastAPI backend, and the PostgreSQL database to be developed with clear documentation and stable outputs at each stage before integrating them with the React Native mobile application. The Waterfall model was chosen because it provides clarity, well-defined milestones, and strong control over the development process, ensuring that each stage is fully completed and validated before moving to the next.

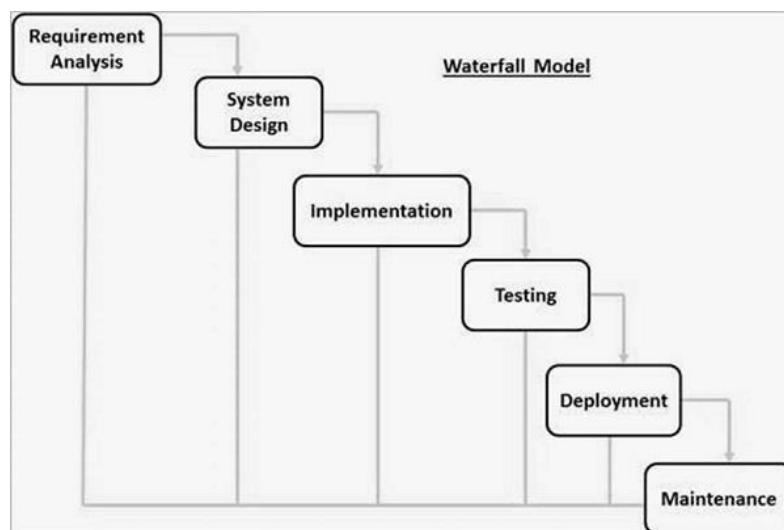


Figure 10 Waterfall Model

3.4.2 Advantages and Disadvantages of the Waterfall Model

3.4.2.1 Advantages

Clear structure and simplicity:

The Waterfall model gives a simple and clear-cut process. As a result, it not only becomes easy to comprehend, but also to manage and document, this is especially true in the case of teams that are not very experienced.

Strong documentation and traceability:

The model practically guarantees a paper trail through the formal documentation generated at each stage. This, in turn, facilitates knowledge transfer, enhances maintenance in the long run, and allows for better traceability.

Effective for stable, well-defined requirements:

The model excels in situations where the user requirements are completely known and will not change during the development process.

Predictable timelines and deliverables:

Since each phase cannot begin until the preceding one has been finished, it is quite simple for managers to follow the development, foresee the delivery dates and pinpoint the milestones.

Suitable for small-scale or low-risk projects:

Waterfall may prove to be a time and cost-saving approach when applied to projects that are not very complex or uncertain.

3.4.2.2 Disadvantages

Lack of flexibility:

The inflexible nature of the model makes it hard to revisit a previous phase and to incorporate changes there. As a result, it leads to delays and higher costs for the project.

Poor adaptability to changing requirements:

If the user needs a change during development, the Waterfall model does not handle these changes well. This can result in a final product that does not fully match user expectations.

Late detection of issues:

Testing occurs only after implementation is finished. This means major problems may not be discovered until the end of the project, making them more expensive and time-consuming to fix.

3.4.3 Development Process Steps

Requirement Analysis:

In the requirement analysis phase, the team reviewed existing marketplace platforms and identified the main functions of TUMB (registration, posting, search, comparison). These functional and non-functional requirements were documented and prioritized.

System Design:

In the system design phase, the overall architecture was defined as a React Native mobile application connected to a FastAPI backend with a PostgreSQL database. UML diagrams, including use case, sequence, and entity diagrams, were created to describe the main user interactions and the data structure for users and listings. The rule-based chatbot was designed as a set of conversation flows for key tasks, such as registration, posting, search, comparison, and recommendations, with clear input validation, error messages, and transitions between each step.

Implementation:

In the implementation phase, the system design was translated into working code. The backend was first developed using FastAPI, with endpoints for user registration and login, creating and managing item listings, and searching and comparing items. Then, the PostgreSQL collections for users and listings were created and connected to these endpoints. After that, the rule-based chatbot was implemented to call the backend APIs and guide the user step by step through tasks such as registration, posting, and searching. Finally, the React Native mobile application was developed to provide a chat interface and simple screens for viewing items and comparison results, and all components were tested together to ensure correct integration.

Testing:

In the testing phase, the main functions of the system were verified to ensure they worked correctly. Unit tests were applied to key backend features, such as registration, login, posting items, and searching. Integration testing was then used to check that the chatbot, FastAPI backend, and PostgreSQL database worked together as expected. Scenario-based tests with sample student data were also carried out to simulate typical actions like registering, posting an item, and comparing products, and any issues found were corrected.

Evaluation and Refinement:

Based on the results of testing and simulated student feedback, chatbot rules and responses were refined to improve clarity, accuracy, and interaction flow.

System Modeling:

In the system modeling phase, the team represented the structure and workflow of the TUMB system to ensure clear understanding of its processes and interactions. The system's components, user actions, and data flows were modeled to guide implementation and testing. This phase followed the Waterfall Model–Design approach, providing a structured blueprint for development and ensuring that all processes were well-defined before coding began.

3.4.4 Constraints and Resources

The project was limited by time, technical resources, and access to real user data. For initial testing, simulated item listings and sample student profiles were used. All tools and frameworks were open-source, chosen for their performance, scalability, and compatibility with the project's academic scope.

3.5 Data Collecting and Processing

The Taibah University Marketplace Bot (TUMB) system did not rely on an external dataset. Instead, data was collected and generated internally during the system's testing and development stages. The data mainly consisted of sample student accounts and mock item listings created to simulate real marketplace usage within the university environment.

During development, these datasets were used to test the chatbot's ability to process user inputs, manage listings, and handle searches. Each listing included attributes such as item name, description, price, image, and category (e.g., books, electronics, clothing, or lab equipment).

The chatbot processed user queries by matching them to predefined rules and retrieving the relevant data from the **PostgreSQL database** through **FastAPI** endpoints. This allowed the team to verify that the chatbot could correctly interpret student inputs, filter results, and display the right items based on category or price range.

For privacy and security reasons, **no real student information** was used during testing. All input and output data were generated by the project team solely for development and demonstration purposes.

3.6 Summary

This chapter presents the **system analysis phase** of the Taibah University Marketplace Bot (TUMB) project. It began with an overview of existing marketplace systems in Saudi Arabia and identified the gap for a student-focused trading platform. The chapter then detailed the system requirements, including functional, non-functional, and user requirements that define how the application should operate.

It also described the system's use cases, showing the interactions between students, administrators, and the rule-based chatbot. The adopted Waterfall development model was explained, presenting each phase of the process in a structured sequence—from requirements analysis, to design, implementation, testing, and final evaluation. Finally, the section on data collection and processing clarified that simulated datasets were used to test and validate the system's functionality according to the planned development stages.

Overall, this chapter established a clear analytical foundation for the project's next phase — system design, which will define the architecture, database structure, and interface components of the TUMB application.

Chapter4: System Design

4.1 Introduction

This chapter is dedicated to the system design of the Taibah University Marketplace Bot (TUMB) application. We begin with Section 4.2 explaining the architectural structure and component organization within the system. Following this, Section 4.3 explores the object-oriented design, which is further broken down into the structural static models (4.3.1) and dynamic models (4.3.2) to clarify the system's internal structure and behavioral sequences. The chapter concludes with an overview of the data and user interface design in subsequent sections.

4.2 Architectural Design

Architectural Design defines the structural framework of a system, specifying its components, their relationships, and how they interact to meet functional requirements. This project adopts a Client-Server model integrated with Clean Architecture to ensure high modularity. The React Native client interacts with a FastAPI backend via RESTful HTTP APIs, where the server is organized into four layers: API for routing, Application for flow orchestration, Domain for core business rules (Rule Engine), and Persistence for PostgreSQL data management. This architectural design provides a clear separation of concerns across system components.

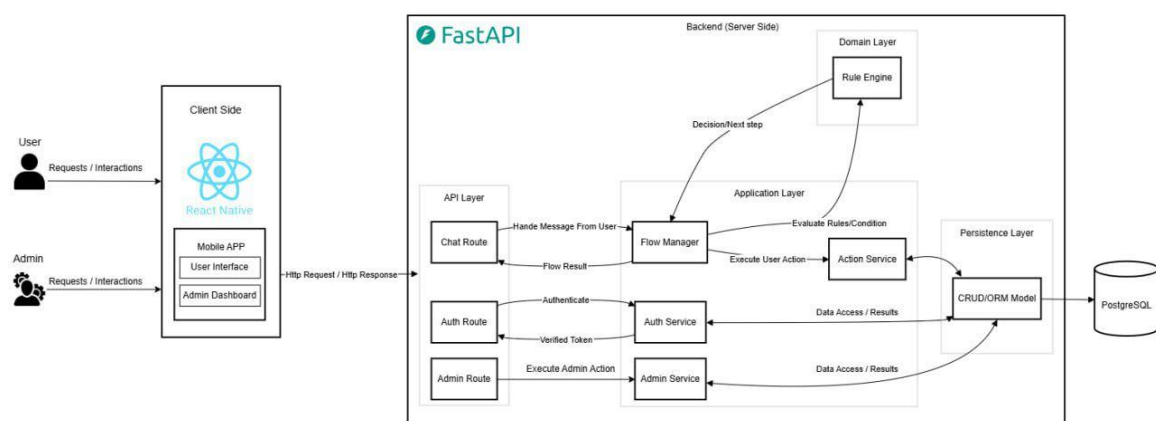


Figure 11 Architectural Diagram

4.3 Object Oriented Design

This segment focuses on the Object-Oriented Design (OOD) methodology employed to structure the TUMB system, utilizing interacting software objects to encapsulate data and behavior, thereby enhancing modularity and maintainability.

4.3.1 Structural Static Models

The system's static structure is formally documented using a Class Diagram. This model is crucial for visualizing the core entities, their properties (attributes), the operations they can perform, and the static relationships that exist between them.

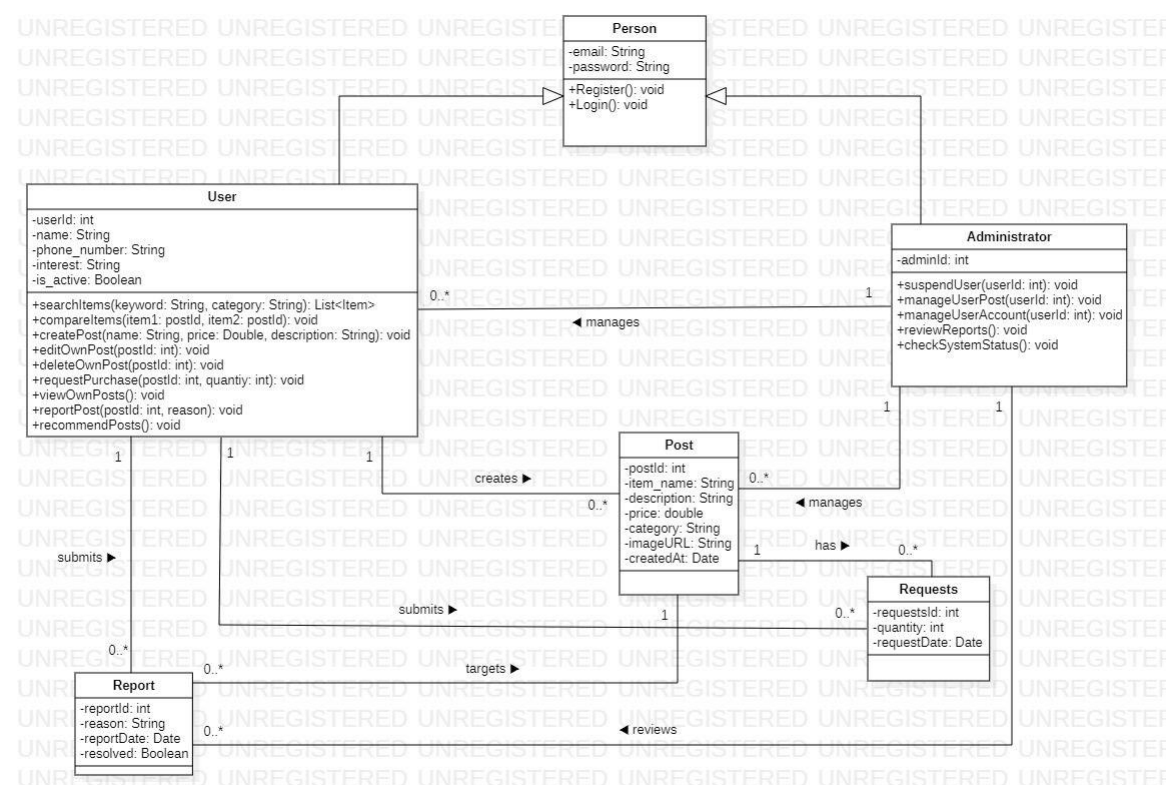


Figure 12 Class Diagram

4.3.2 Dynamic Models

To illustrate the behavior and sequence of events during runtime, Dynamic Models are employed, primarily in the form of Sequence Diagrams. These diagrams depict the time-ordered flow of messages and interactions between the actors (users/admins) and the system's objects (e.g., Chatbot Interface, Backend Service, Database).

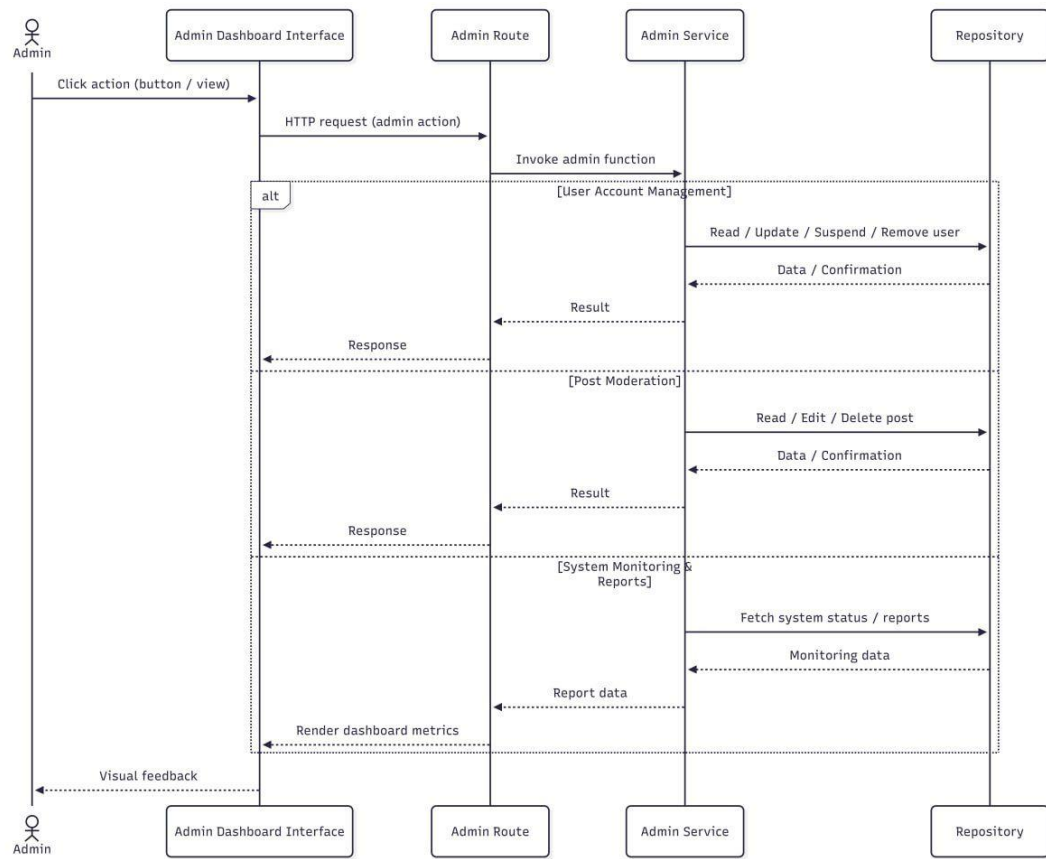


Figure 13 Admin Sequence Diagram

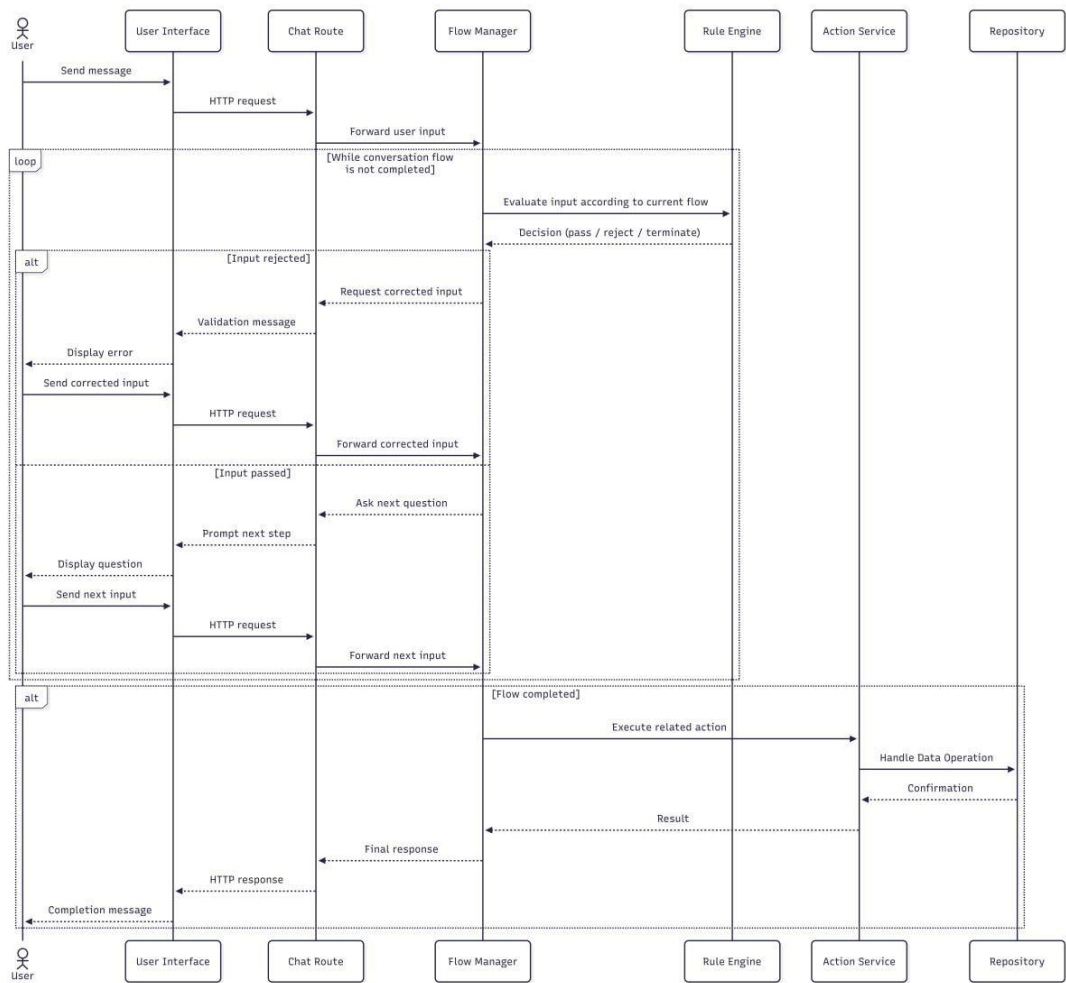


Figure 14 User Sequence Diagram

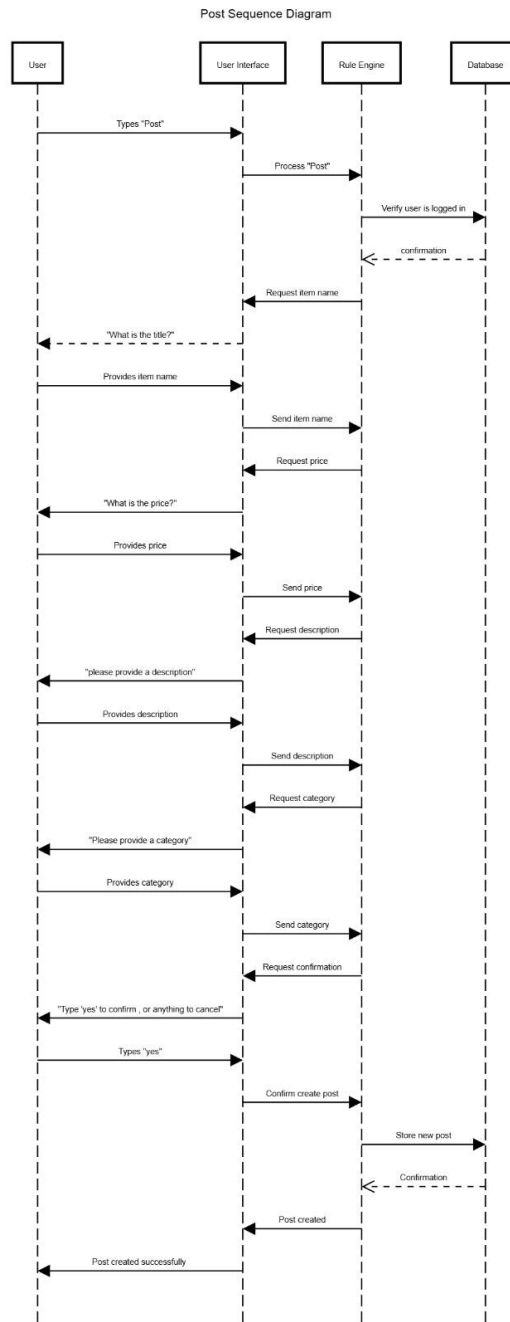


Figure 15 Post Sequence Diagram

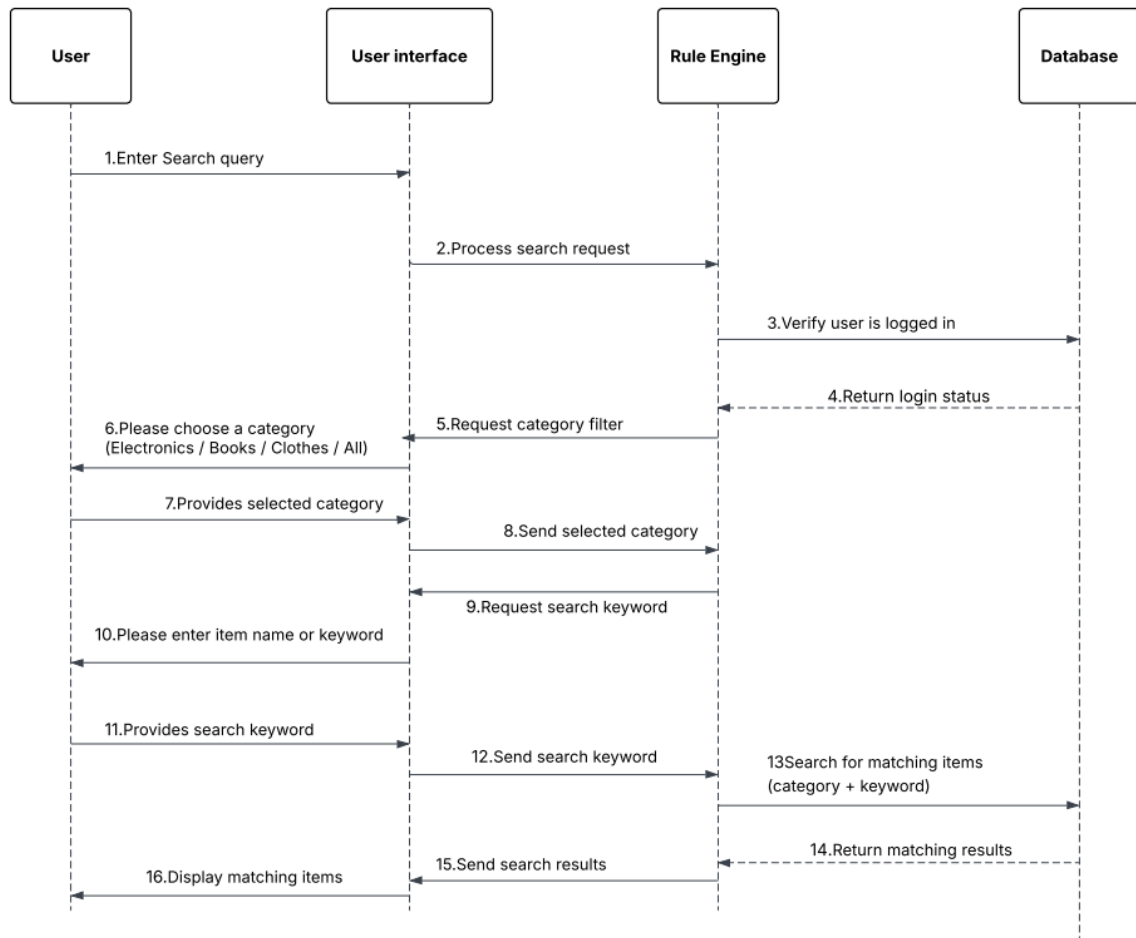


Figure 16 Search Sequence Diagram

Admin System Monitoring Sequence Diagram

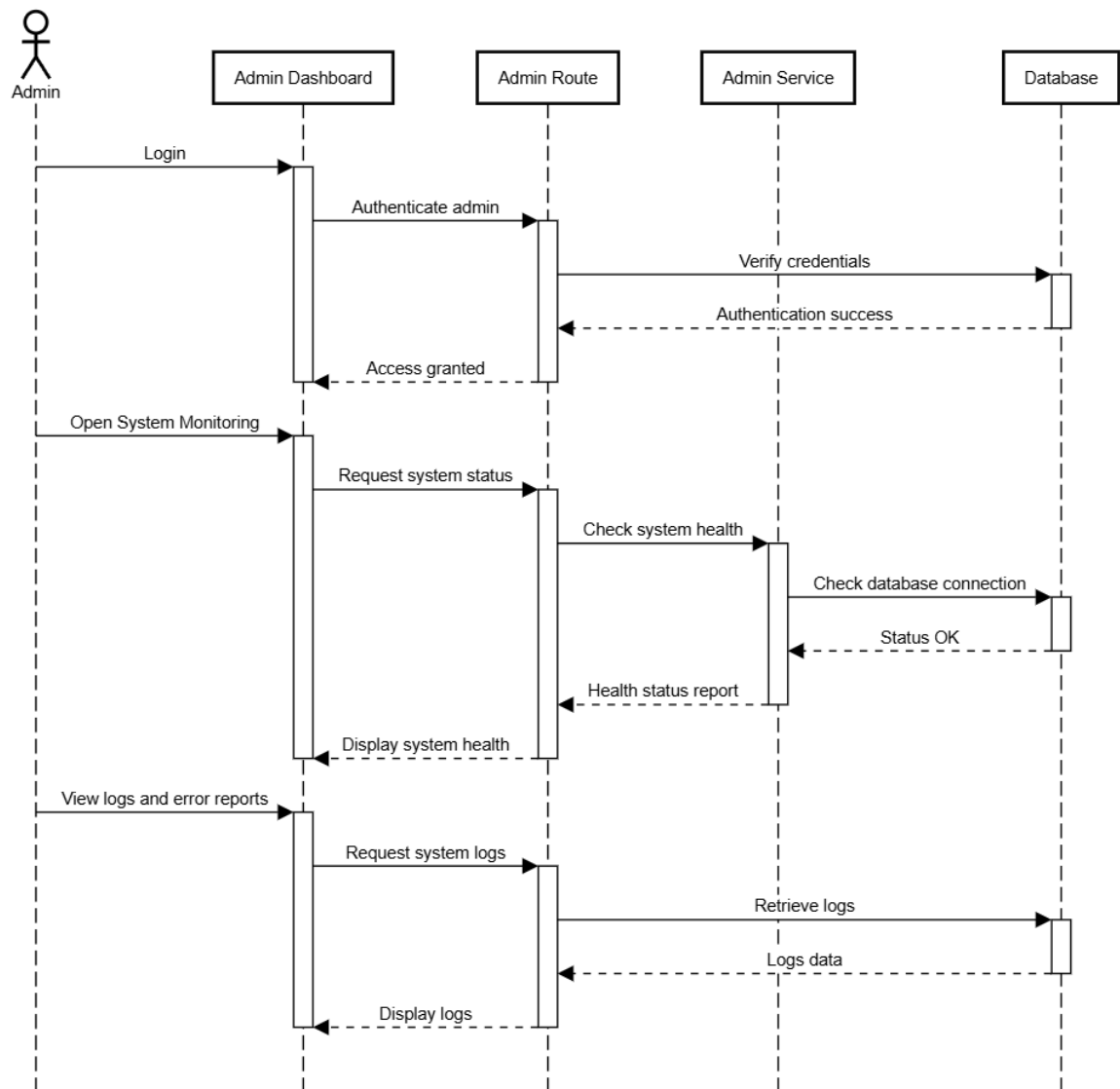


Figure 17 Monitoring Sequence Diagram

4.4 Data Modeling

Data modeling defines how data is organized and stored within the system, attributes, data type, description and constraints are presented.

4.4.1 Database Design

The database is designed using multiple related tables connected through primary and foreign keys to support system functionality.

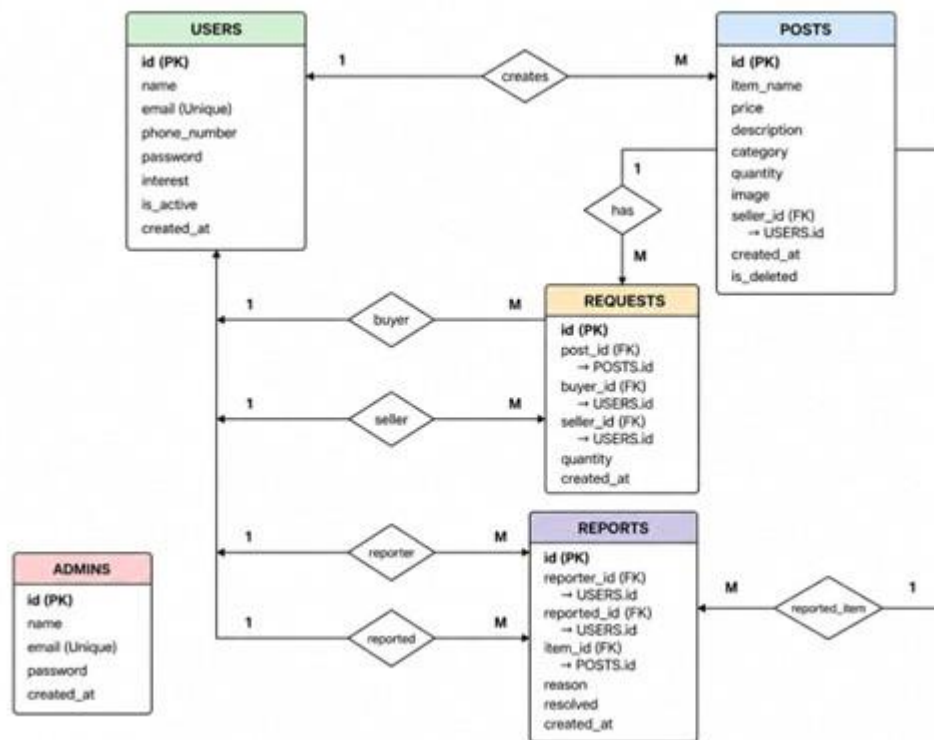


Figure 18 ER Diagram

4.4.2 Relational Schema

The relational schema describes the structure of the database, including tables, attributes and the relationships between them.

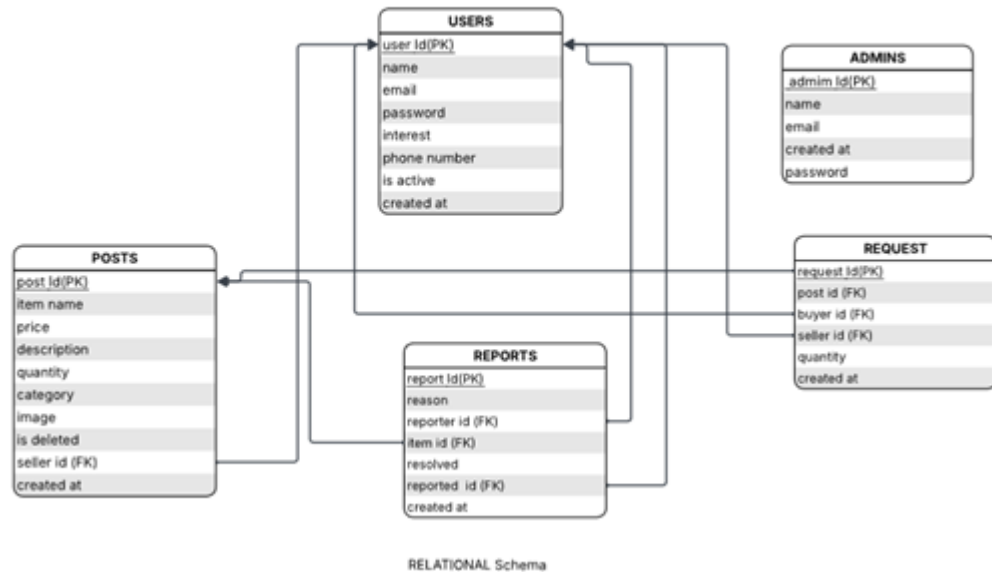


Figure 19 Relational schema

4.4.3 Database Table

This section presents the main database tables.

4.4.3.1 Users Table

Attribute	Data Type	Description	Constraints
User id	INT	Unique identifier for each user	Primary Key, INDEX
Name	VARCHAR	User full name	NOT NULL
Email	VARCHAR	User email address	UNIQUE,NOT NULL, INDEX
Password	VARCHAR	user password	NOT NULL
Interest	VARCHAR	User interest c	NOT NULL
Phone number	VARCHAR	User phone number	NOT NULL
Is active	BOOLEAN	User account status	DEFAULT TRUE
Created at	TIMESTAMP	Account creation date	DEFAULT CURRENT_TIMESTAMP

Table 24 Users Table

4.4.3.2 Posts Table

Attribute	Data Type	Description	Constraints
ID	INT	Unique identifier for each post	Primary Key, INDEX
Item name	VARCHAR	Name of the item	NOT NULL
Price	INT	item Price	NOT NULL
quantity	INT	Available quantity	NOT NULL,DEFAULT1
Description	VARCHAR	item Description	NOT NULL
Category	VARCHAR	Item category	INDEX ,NOT NULL
Image	VARCHAR(255)	Image path or URL	NULL
seller_id	INT	Seller User ID	Foreign Key → USERS(user_id) ,ON DELETE CASCADE,INDEX ,NOT NULL
Created at	TIMESTAMP	Post Created data	DEFAULT,CURRENT_TIMESTAMP
IS Deleted	BOOLEAN	Soft delete status	DRFAULT FALSE,INDEX ,NOT NULL

Table 25 Posts Table

4.4.3.3 Report Table

Attribute	Data Type	Description	Constraints
ID	INT	Unique identifier for each report	Primary Key, INDEX
Reported ID	INT	USER being reported	Foreign Key → USER(id), INDEX,NOT NULL
Reason	VARCHAR	Reason for report	NOT NULL
Reporter id	INT	User who reported	Foreign Key → USERS(user_id)
Item id	INT	Reported post ID	Foreign Key → POSTS(post_id)
Resolved	VARCHAR	Report status	DEFAULT FALSE
Created at	TIMESTAMP	Report Created date	DEFAULT,CURRENT_TIMESTAMP

Table 26 Reports Table

4.4.3.4 Admins Table

Attribute	Data Type	Description	Constraints
admin_id	INT	Unique identifier for admin	Primary Key, INDEX
name	VARCHAR(100)	Admin name	NOT NULL
email	VARCHAR(100)	Admin email	NOT NULL, INDEX, UNIQUE
password	VARCHAR(100)	Encrypted password	NOT NULL
Created at	TIMESTAMP	Admin account creation date	DEFAULT CURRENT_TIMESTAMP

Table 27 Admins Table

4.4.3.5 Requests Table

Attribute	Data Type	Description	Constraints
ID	INT	Unique identifier for request	Primary Key, INDEX
Post ID	INT	Requested Post ID	Foreign Key → posts(id), ON DELETE CASCADE, INDEX , NOT NULL
Buyer ID	INT	Admin email	Foreign Key → Users(id), INDEX, NOT NULL
Seller ID	INT	Encrypted password	Foreign Key → Users(id), INDEX, NOT NULL
Quantity	INT	Request quantity	NOT NULL, DEFAULT 1
Created at	TIMESTAMP	Admin account creation date	DEFAULT CURRENT_TIMESTAMP

Table 28 Requests Table

4.5 User Interface Design

The user interface is designed to be simple and easy to use. It allows users to interact with the system easily and perform tasks.

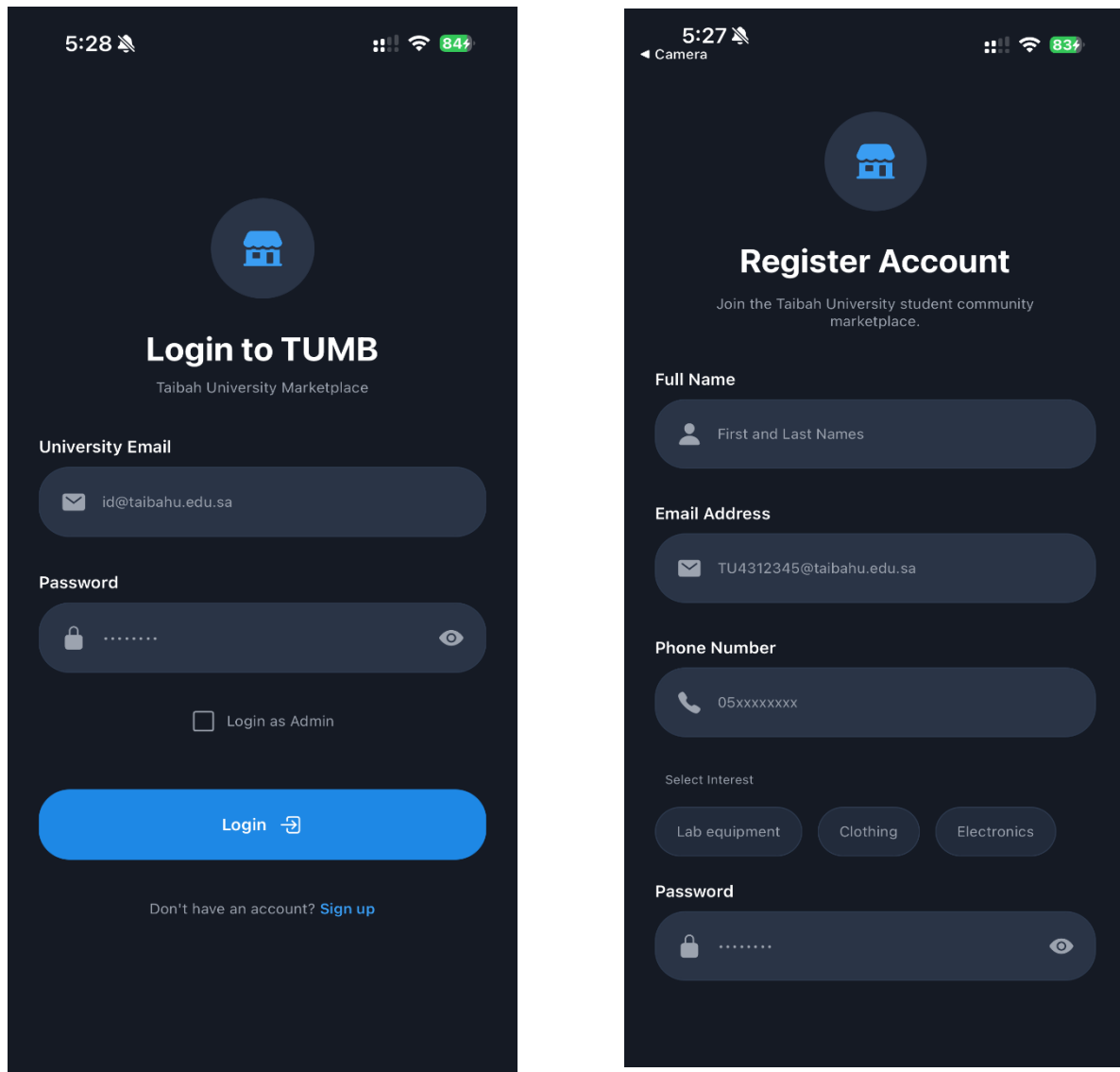


Figure 20 Login and Registration

The Login and Registration interface allows users to access the system or create a new account.

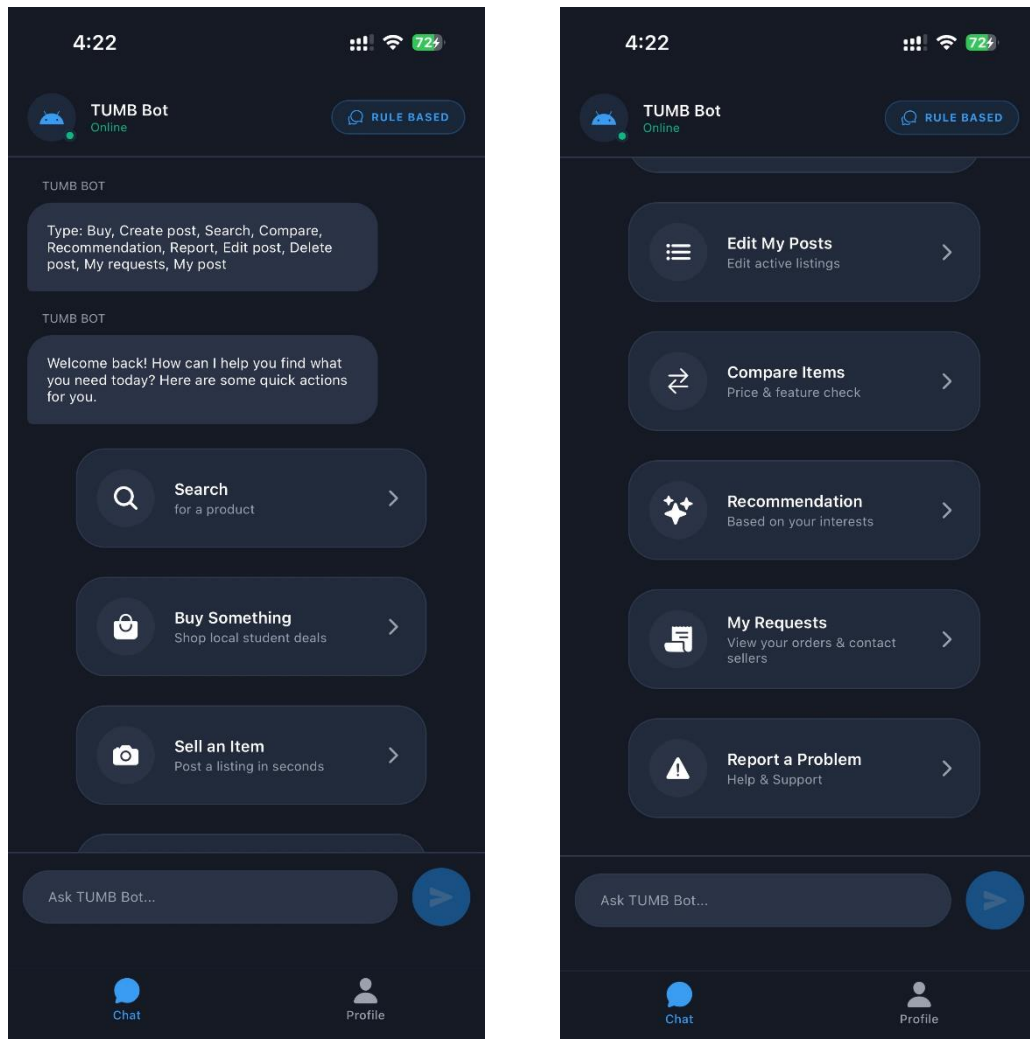


Figure 21 Chat

The Chat interface includes a rule-based chatbot that helps users to interact with the system through guided conversation.

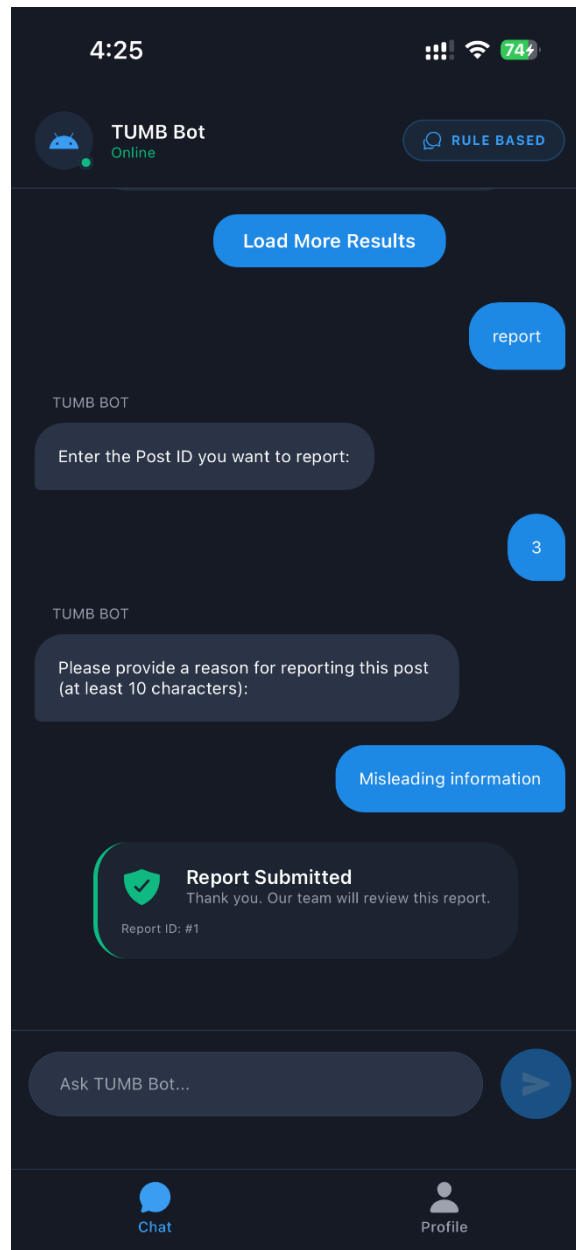


Figure 22 Reports

The Report interface allows users to report inappropriate behavior, content, or other users within the system.

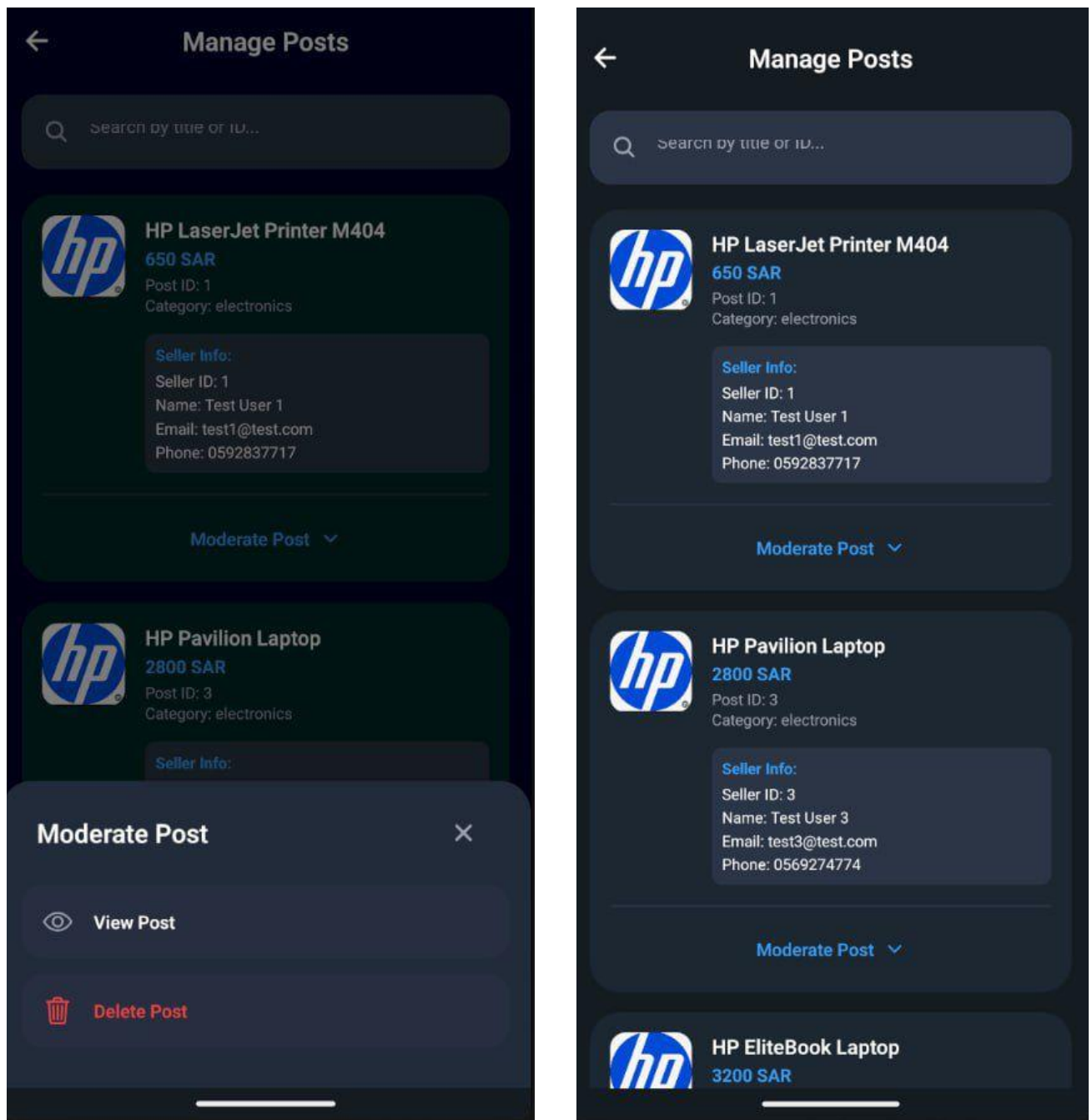


Figure 23 Mange Posts (reports)

The Manage Posts interface allows administrators to review, track, and act on reports submitted by users.

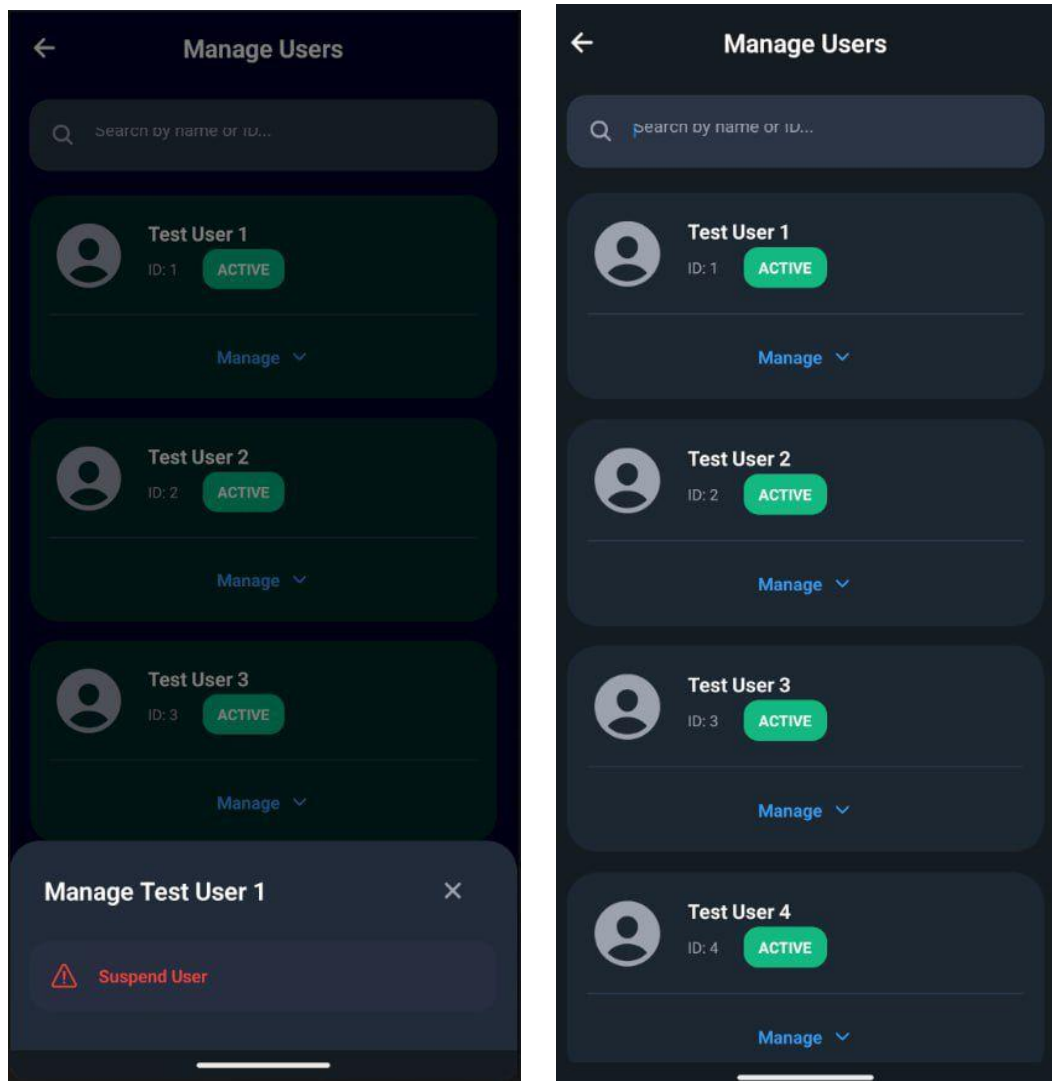


Figure 24 Admin Management

The Managing Users & Posts interface allows administrators to monitor, control, and manage user accounts.

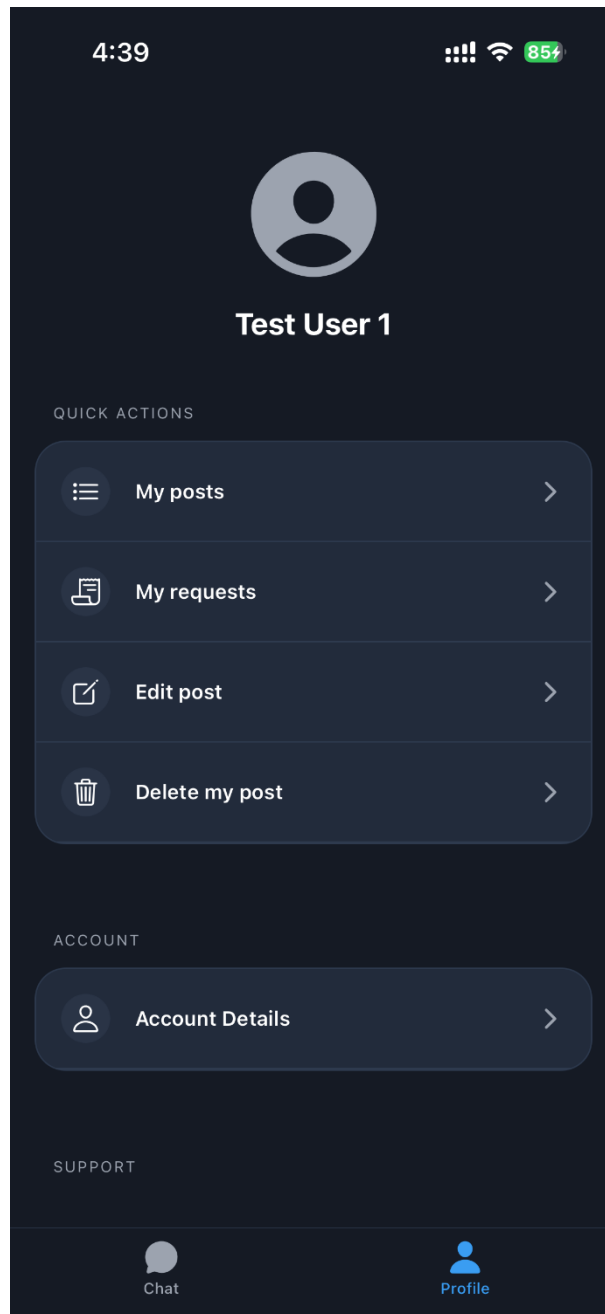


Figure 25 User & Admin Profile

The Admin and User Profile interface allows both administrators and users to manage their information and system activities.

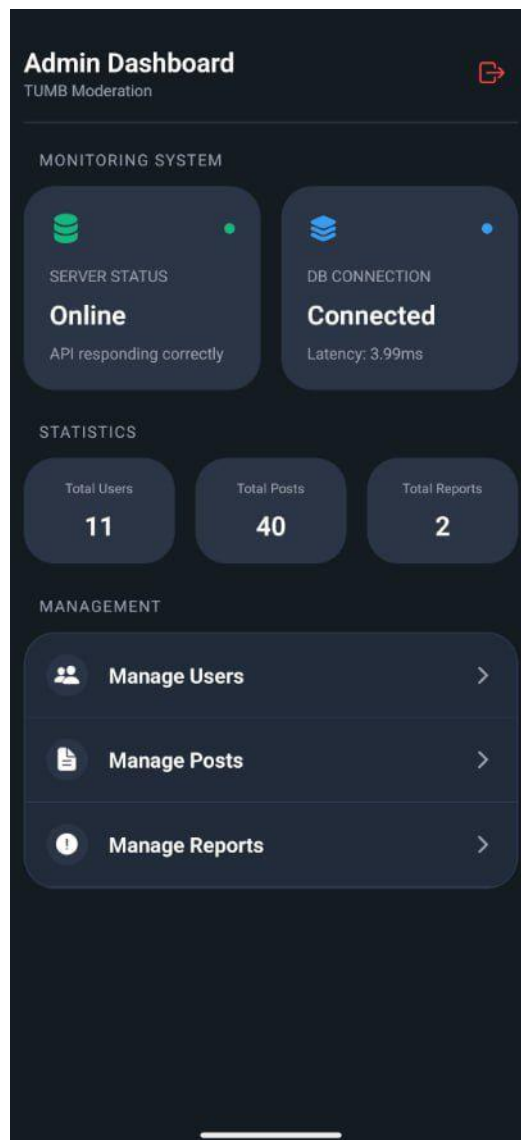


Figure 26 Admin Dashboard

The Admin Dashboard interface provides administrators with control and monitoring of the system.

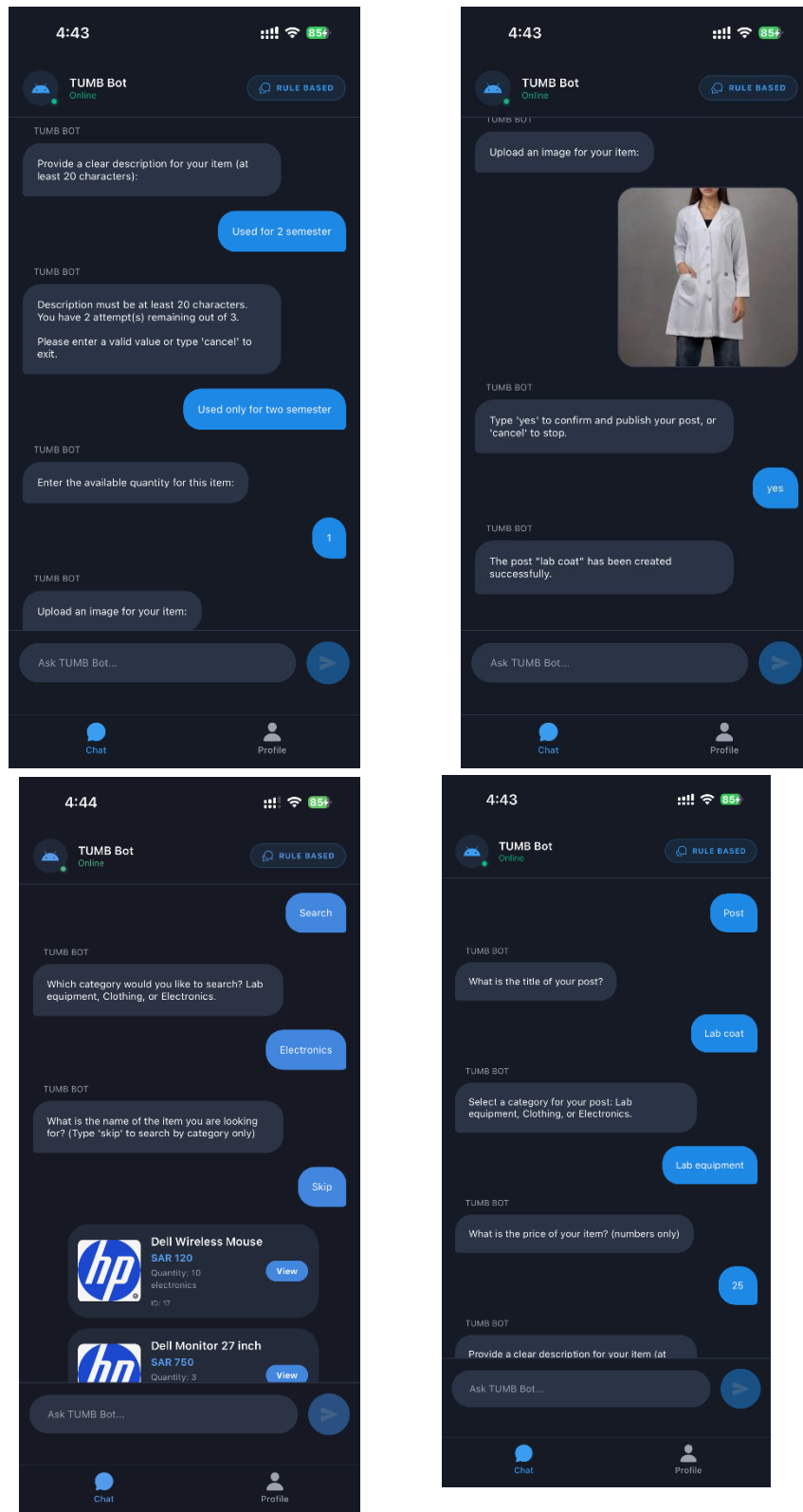


Figure 27 Post & Search

The Search and Post interface allows users to find items and create new posts for buying or selling. It provides a simple and organized layout.

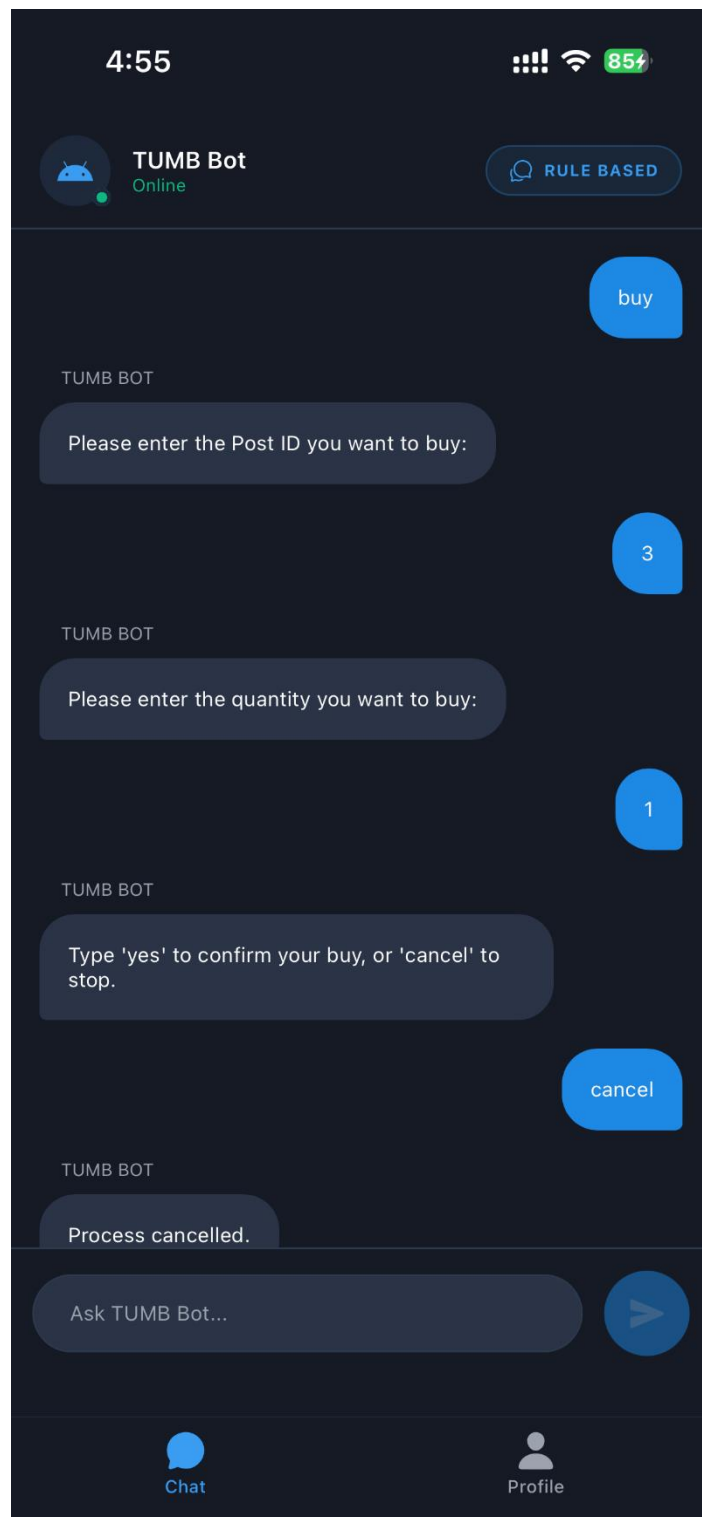


Figure 28 Buy

The Buy interface allows users to purchase items through a clear and guided process.

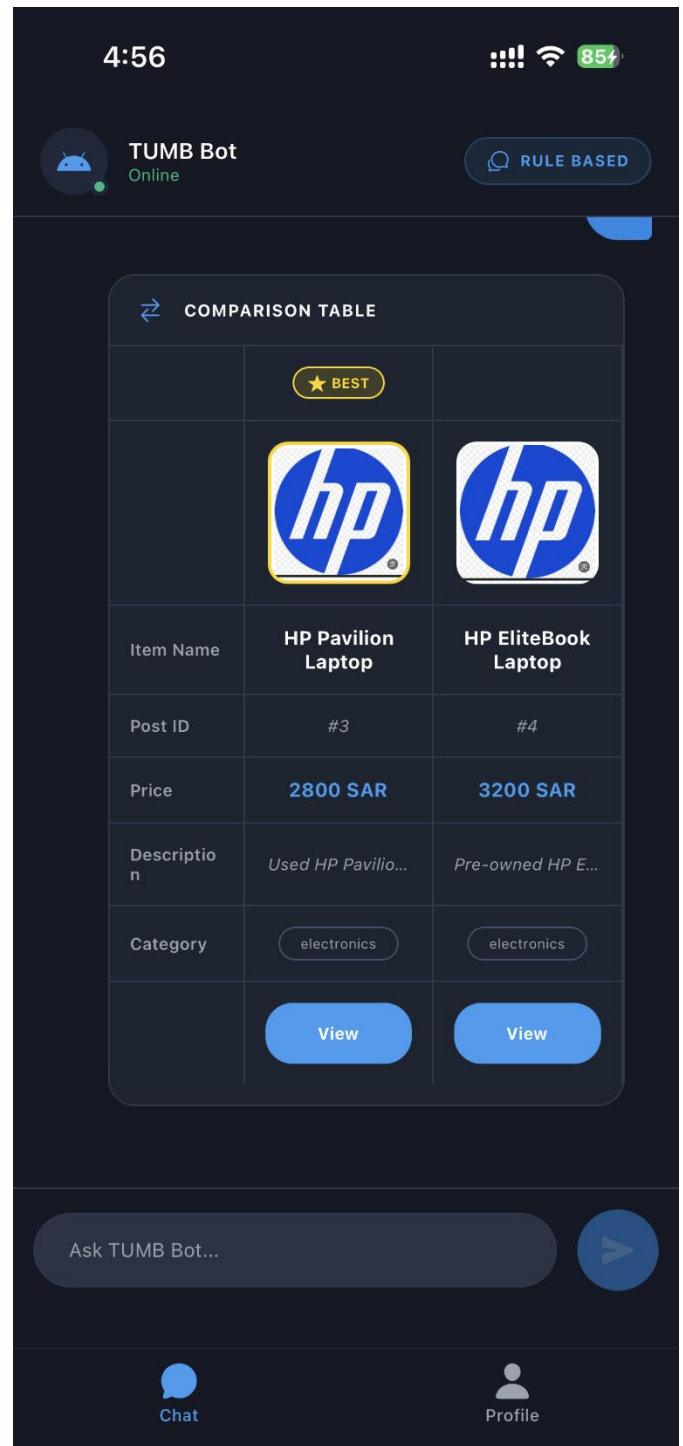
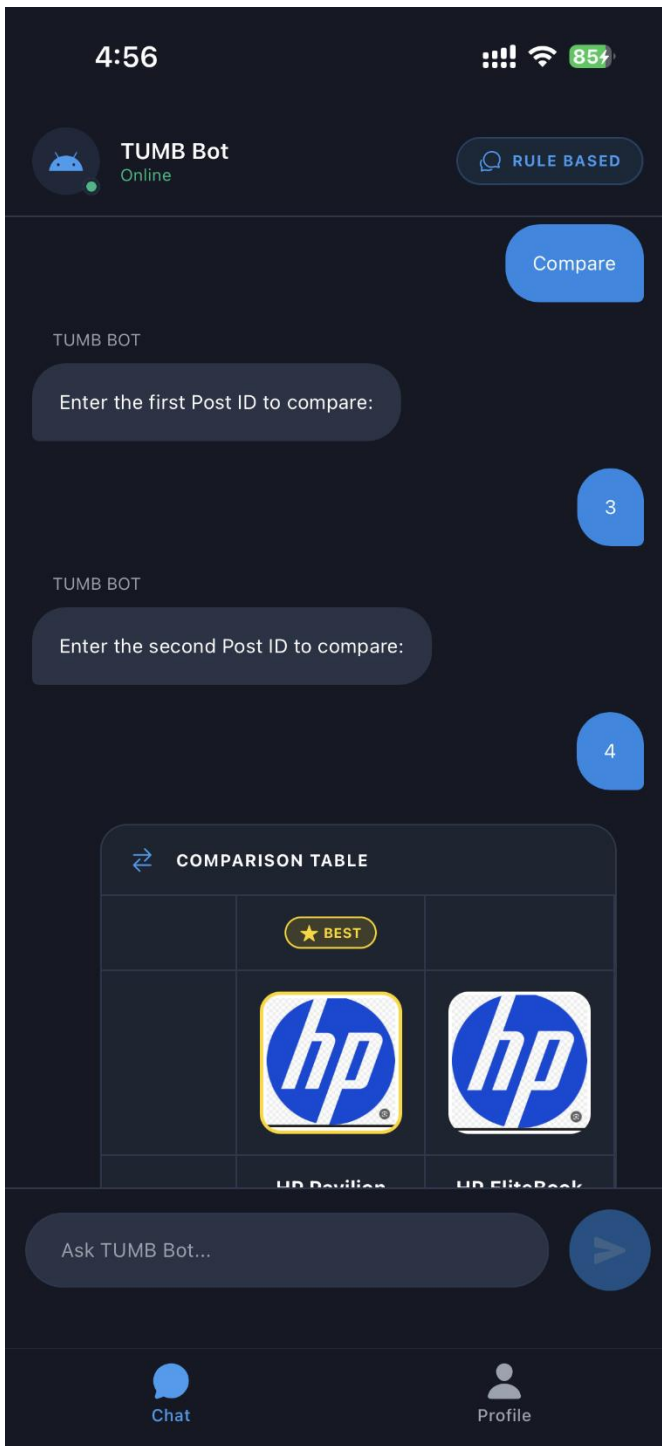


Figure 29 Compare

The Compare interface allows users to compare multiple items side-by-side to support better decision-making.

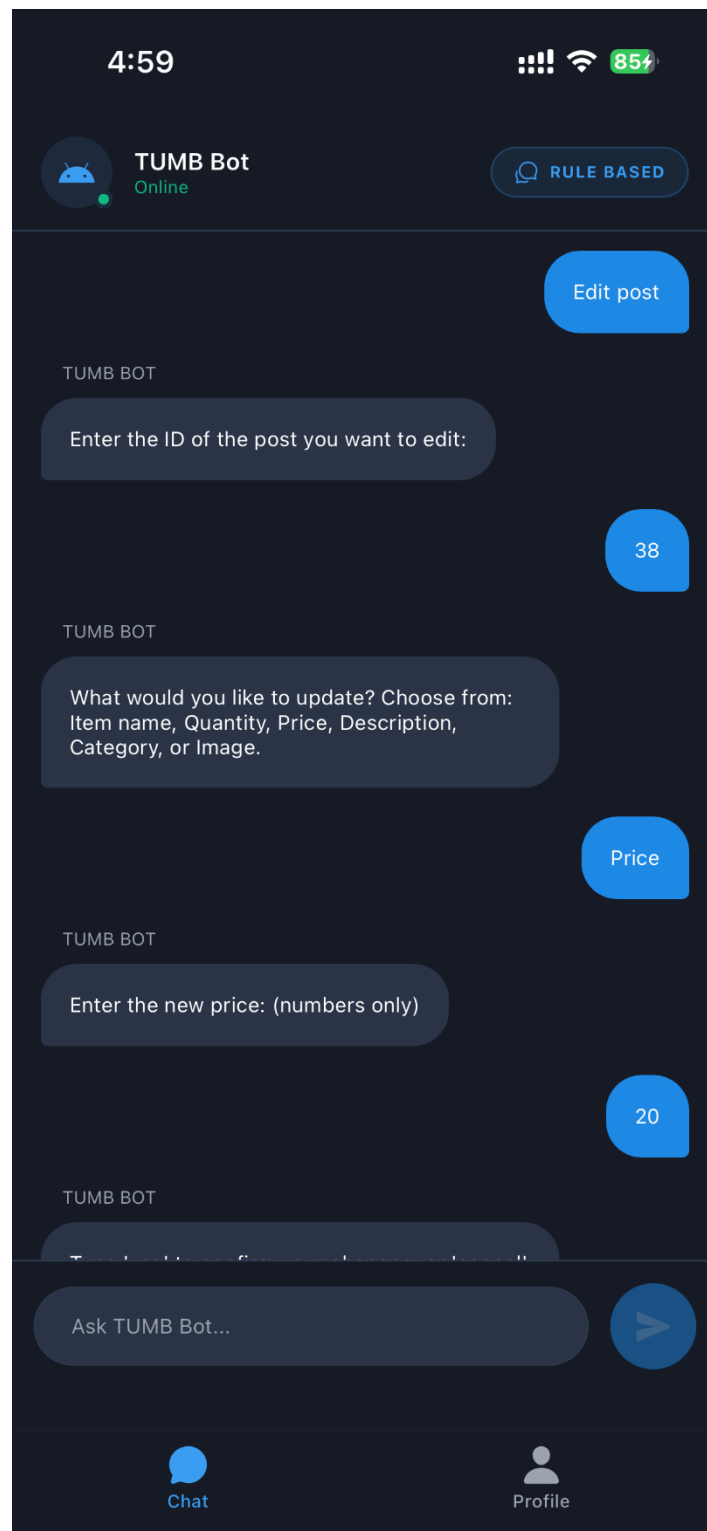


Figure 30 Mange

The Manage Listing interface allows users to control and organize their posted items.



Figure 31 Recommendation

The Recommendation interface suggests relevant items to users based on their browsing history.

4.6 Summary

This chapter presented the system design of the project, including architectural design, object-oriented design, data modeling and user interface design. The presented designs serve as a blueprint for the implementation phase of the system.

Chapter 5: System Implementation

5.1 Introduction

This chapter presents the implementation phase of the Taibah University Marketplace Bot (TUMB) application. It demonstrates how the architectural design, UML diagrams, database schema, and system models described in Chapter 4 were systematically translated into a fully functional software system. The implementation strictly follows the layered Clean Architecture model adopted during the design phase. Each architectural component was developed as an independent logical layer to ensure modularity, separation of concerns, scalability, and maintainability.

5.2 Tools and Languages

This section presents the core technologies used in developing the TUMB application. These technologies were carefully selected to support the layered Clean Architecture model and ensure efficient communication between the frontend, backend, and database components.

Tool / Language	Role in System	Justification
React Native	Frontend (Mobile Application)	Used to develop a cross-platform mobile application with a single codebase for Android and iOS, ensuring consistent user experience and reduced development effort.
FastAPI	Backend Framework	Provides high-performance RESTful API services, structured routing, automatic validation, and scalable request handling.
Python	Backend & Rule Engine Logic	Used to implement business logic and the rule-based chatbot engine using structured conditional logic and state management.
PostgreSQL	Relational Database	Stores structured data such as users, posts, reports, and admin records with enforced relational integrity through primary and foreign keys.

RESTful API	Client–Server Communication	Enables structured and standardized communication between the React Native frontend and FastAPI backend using HTTP methods and JSON responses.
-------------	-----------------------------	--

Table 29 Core Tools and Languages Used in TUMB

5.3 Mapping Design to Implementation

This section explains how the layered Clean Architecture described in Chapter 4 was practically implemented. Each architectural layer — API, Application, Domain, and Persistence — was developed as a separate logical component to ensure clear separation of concerns and maintain system modularity.

5.3.1 Client–Server Architecture Implementation

As defined in Chapter 4, the system adopts a Client–Server architecture. The React Native mobile application acts as the Client, while FastAPI represents the Server. Communication between the two components is handled through RESTful HTTP requests. The client sends structured requests such as login, search, post creation, or comparison, and the server processes these requests before interacting with the PostgreSQL database.

5.3.2 API Layer Implementation

The API Layer was implemented using FastAPI routing endpoints such as /register, /login, /search, /compare, /report, and /recommendations. Each endpoint validates incoming data, ensures proper authentication where required, and forwards the request to the Application Layer.

5.3.3 Application Layer Implementation

The Application Layer orchestrates use case execution and manages workflow logic, including user registration, login verification, post creation, search filtering, comparison operations, and recommendation generation. This layer ensures that system behavior follows the sequence diagrams and use case definitions presented in Chapter 4.

5.3.4 Domain Layer (Rule Engine)

The Domain Layer contains the core business logic of the system, including the rule-based chatbot engine. It was implemented in Python using structured decision rules,

conditional statements, and state tracking mechanisms. The system verifies authentication before allowing actions and ensures required fields are completed before accepting new posts. This implementation directly reflects the Rule Engine component defined in the architectural design in Chapter 4.

5.3.5 Persistence Layer Implementation

The Persistence Layer is responsible for database interaction and data storage. PostgreSQL tables (USERS, POSTS, REPORTS, ADMINS) were implemented according to the ER Diagram and Relational Schema presented in Chapter 4. Primary and foreign key constraints were enforced to maintain data integrity. All CRUD operations are executed through this layer to ensure secure and structured data management.

5.4 Main/Most Important Codes

```
1  async def register_user(self, user) -> dict:
2
3      email_error = validate_university_email(user.email)
4      if email_error:
5          raise ApplicationError(email_error, status_code=422)
6
7      phone_error = validate_phone_number(user.phone_number)
8      if phone_error:
9          raise ApplicationError(phone_error, status_code=422)
10
11     pass_error = get_password_errors(user.password)
12     if pass_error:
13         raise ApplicationError("\n".join(pass_error), status_code=422)
14
15     existing = await self.user_repo.get_by_email(user.email)
16     if existing:
17         raise ApplicationError("Email already registered", status_code=409)
18
19     hashed_password = hash_password(user.password)
20     user_data = {
21         "name": user.name,
22         "email": user.email,
23         "phone_number": user.phone_number,
24         "password": hashed_password,
25         "interest": user.interest
26     }
27
28     otp = generate_otp()
29     await save_pending_user(user.email, user_data, otp)
30
31     email_service = EmailService()
32     _email_sent_task = asyncio.create_task(email_service.send_otp_email(user.email, otp))
33
34     return {"message": "OTP sent to email", "expires_in": 180}
```

Figure 32 Register User Function

This function handles new user registration by first validating the email, phone number, and password, raising a 422 error for any invalid input. It also checks if the email is already registered and raises a 409 error to prevent duplicates. If all checks pass, it hashes

the password and builds the user data object. Instead of saving the user directly to the database, it stores them as pending with a generated OTP, then asynchronously sends that OTP to their email. Finally, it returns a success message confirming that the OTP was sent, along with the time the OTP remains valid which is 180 seconds.

```
1     async def login_user(self, email: str, password: str) -> dict:
2         user = await self.user_repo.get_by_email(email)
3         if not user or not verify_password(password, user.password):
4             raise ApplicationError("Incorrect email or password", status_code=401)
5
6         if not user.is_active:
7             raise ApplicationError("Your account has been suspended", status_code=403)
8
9         token = create_access_token({"sub": user.email, "type": "user"})
10        return {
11            "message": "User login successful",
12            "access_token": token,
13        }
```

Figure 33 Login User Function

This function handles user authentication. It looks up the account associated with the provided email, then verifies that the password is correct. If either the email or password is wrong, it raises an unauthorized error. It also checks whether the account is currently active, and if the account has been suspended, it raises a separate error to inform the user. If all checks pass, it generates an access token that the user can use to authenticate future requests, and returns it along with a success message.

```
1     async def create_post(self, ctx: ActionContext) -> dict:
2         # Creates a new post using the data collected from the user during the flow.
3         # After creation, we call save() to commit the transaction.
4
5         new_post = await self.post_repo.create_post(
6             item_name=ctx.data.get("item_name"),
7             category=ctx.data.get("category"),
8             price=int(ctx.data.get("price")),
9             description=ctx.data.get("description"),
10            quantity=int(ctx.data.get("quantity")),
11            image=ctx.data.get("image"),
12            seller_id=ctx.user_id
13        )
14        await self.post_repo.save()
15
16        return {
17            "id": new_post.id,
18            "item_name": new_post.item_name,
19            "category": new_post.category,
20            "price": new_post.price,
21            "seller_id": new_post.seller_id
22        }
```

Figure 34 Create Post Function

This function handles creating a new marketplace listing. It takes the item details from the context data, such as name, category, price, description, quantity, and image, along with

the seller's identity, and saves the post to the database. It then returns the key details of the newly created post including its ID, name, category, price, and seller ID.

```
1  async def compare_posts(self, ctx: ActionContext) -> dict:
2      # Fetches two posts by their IDs (provided in ctx) and compares them.
3      # The best_choice is simply the post with the lower price.
4      p1 = await self.post_repo.get_by_id(int(ctx.data.get("post_id_1")))
5      p2 = await self.post_repo.get_by_id(int(ctx.data.get("post_id_2")))
6
7      best_post = p1 if p1.price < p2.price else p2
8
9      return {
10         "post1": {"id": p1.id, "item_name": p1.item_name, "category": p1.category,
11                  "description": p1.description, "price": p1.price, "image": p1.image},
12         "post2": {"id": p2.id, "item_name": p2.item_name, "category": p2.category,
13                  "description": p2.description, "price": p2.price, "image": p2.image},
14         "best_choice": {
15             "id": best_post.id,
16             "item_name": best_post.item_name,
17             "price": best_post.price,
18         }
19     }
```

Figure 35 Compare Post Function

This function takes two post IDs and fetches both from the database. It then compares their prices and identifies the cheaper one as the best choice. It returns the full details of both posts along with a recommendation indicating which one offers the better value, giving the user a clear basis for their decision.

```

1  async def search_posts(self, ctx: ActionContext) -> dict:
2      # Searches posts based on the filters provided by the user.
3      # We use cursor-based pagination: we fetch (limit + 1) posts,
4      # and if we get more than (limit), we know there are more pages.
5      # The next_cursor is the ID of the last displayed post, used to fetch the next page.
6      # This makes the app faster and prevents the interface from slowing down.
7      limit = 4
8
9      keyword = ctx.data.get("item_name")
10
11     posts = await self.post_repo.search(
12         user_id=ctx.user_id,
13         name=None if keyword == "skip" else keyword,
14         category=ctx.data.get("category"),
15         last_id=ctx.data.get("last_id"),
16         limit=limit + 1
17     )
18     has_more = len(posts) > limit
19     display_posts = posts[:limit]
20     next_cursor = display_posts[-1].id if display_posts and has_more else None
21
22     return {
23         "posts": [
24             {
25                 "id": p.id,
26                 "item_name": p.item_name,
27                 "category": p.category,
28                 "price": p.price,
29                 "quantity": p.quantity,
30                 "description": p.description,
31                 "seller_name": p.seller.name,
32                 "seller_phone": p.seller.phone_number,
33                 "seller_email": p.seller.email,
34                 "image": p.image,
35             }
36             for p in display_posts
37         ],
38         "count": len(display_posts),
39         "has_more": has_more,
40         "next_cursor": next_cursor,
41         "filters": {
42             "item_name": ctx.data.get("item_name"),
43             "category": ctx.data.get("category")
44         }
45     }

```

Figure 36 Search Post Function

This function handles searching for posts based on optional filters like item name and category. It also supports cursor-based pagination, where the client can pass a `last_id` to indicate the last post it received, so the function knows where to continue fetching from rather than always starting from the beginning. To detect whether more results are available, it intentionally fetches one extra result beyond the requested limit. If the number of results exceeds the limit, it knows there is a next page and sets the cursor to the last visible post's ID so the following request can pick up from there. Otherwise, the cursor is set to `None`, signaling that no more results exist. It returns the posts along with the total count, whether more pages exist, the next cursor value, and the active filters that were applied.

```

1  async def report_post(self, ctx: ActionContext) -> dict:
2      # Submits a report against a post.
3      # Before creating the report, we check if the user has already reported
4      # this exact post to prevent duplicate reports.
5      item_id = int(ctx.data.get("report_item_id"))
6      post = await self.post_repo.get_by_id(item_id)
7
8      already_reported = await self.report_repo.has_reported_item(ctx.user_id, item_id)
9      if already_reported:
10         raise ActionError("You have already reported this post")
11
12     new_report = await self.report_repo.create_report(
13         reporter_id=ctx.user_id,
14         reported_id=post.seller_id,
15         item_id=item_id,
16         reason=ctx.data.get("report_reason")
17     )
18     await self.report_repo.save()
19
20     return {"report_id": new_report.id, "item_id": item_id}
21

```

Figure 37 Report Post Function

This function allows a user to report a specific post for a given reason. It first checks whether the user has already reported that item, and if so, raises an error to prevent duplicate reports. Otherwise, it creates a new report record with the reporter ID, seller ID, item ID, and the provided reason, saves it to the repository, and returns the report ID and item ID.

```

1  async def update_post(self, ctx: ActionContext) -> dict:
2      # Updates an existing post with the new values provided by the user.
3      # We cast price to int separately before building the update parameters,
4      # since the user input arrives as a string from the chat flow.
5      post_id = int(ctx.data.get("post_id"))
6
7      update_params = PostUpdate(**ctx.data)
8      updated_post = await self.post_repo.update_post(post_id, update_params)
9      await self.post_repo.save()
10     return {"item_name": updated_post.item_name}
11

```

Figure 38 Update Post

This function updates an existing post using the data provided in the action context. It extracts the post_id and casts it to an integer since it arrives as a string from the chat flow, then builds the update parameters using PostUpdate. It applies the update to the post in the repository, saves the changes, and returns the updated post's item name.

```

1  async def get_recommendations(self, ctx: ActionContext) -> dict:
2      # Fetches personalized post recommendations based on the user's interest.
3      user = await self.user_repo.get_by_id(ctx.user_id)
4      posts = await self.post_repo.get_recommendations(ctx.user_id, user.interest, limit=5)
5      return {
6          "recommendations": [
7              {
8                  "id": p.id,
9                  "item_name": p.item_name,
10                 "category": p.category,
11                 "price": p.price,
12                 "description": p.description,
13                 "quantity": p.quantity,
14                 "image": p.image,
15                 "seller_name": p.seller.name,
16                 "seller_phone": p.seller.phone_number,
17                 "seller_email": p.seller.email,
18             }
19             for p in posts
20         ],
21         "based_on": user.interest,
22         "count": len(posts)
23     }
24

```

Figure 39 Get Recommendations

This function fetches personalized post recommendations based on the user's interest. It retrieves the user by ID, then fetches up to 5 recommended posts matching their interest. It returns a list of posts with full details including item name, category, price, description, quantity, image, and seller contact info, along with the interest the recommendations are based on and the total count of results.

```

1  async def create_request(self, ctx: ActionContext) -> dict:
2      # Sends a buy request from the buyer to the seller for a specific post.
3      # We check for duplicate requests first to prevent the buyer from
4      # sending multiple requests for the same post.
5      # On success, we return the seller's contact info so the buyer can reach out.
6
7      post_id = int(ctx.data.get("post_id"))
8
9      post = await self.post_repo.get_by_id(post_id)
10
11     existing_requests = await self.request_repo.has_requested_post(ctx.user_id, post_id)
12     if existing_requests:
13         raise ActionError(f"You have already sent a request for this post")
14
15     amount = int(ctx.data.get("quantity", 1))
16     success = await self.post_repo.decrement_quantity(post_id, amount)
17     if not success:
18         raise ActionError("Not enough quantity available")
19
20     new_request = await self.request_repo.create_request(
21         post_id=post_id,
22         buyer_id=ctx.user_id,
23         seller_id=post.seller_id,
24         quantity=amount
25     )
26     await self.request_repo.save()
27
28     seller = await self.user_repo.get_by_id(post.seller_id)
29     buyer = await self.user_repo.get_by_id(ctx.user_id)
30
31     email_service = EmailService()
32     _email_task = asyncio.create_task(
33         email_service.send_request_notification(RequestEmailData(
34             seller_email=seller.email,
35             seller_phone=seller.phone_number,
36             seller_name=seller.name,
37             buyer_name=buyer.name,
38             item_name=post.item_name,
39             quantity=amount,
40             price=post.price,
41             request_id=new_request.id,
42         ))
43     )
44
45     return {
46         "request_id": new_request.id,
47         "item_name": post.item_name,
48         "post_id": post.id,
49         "price": post.price,
50         "seller_name": seller.name,
51         "seller_phone": seller.phone_number,
52         "seller_email": seller.email,
53         "quantity": amount
54     }

```

Figure 40 Create Request

This function handles sending a buy request from a buyer to a seller for a specific post. It first retrieves the post by ID, then checks if the buyer has already submitted a request for it, raising an error to prevent duplicates. It also attempts to decrement the available quantity by the requested amount, raising an error if stock is insufficient. If all checks pass, it creates and saves the new request, then asynchronously sends an email notification to the seller with the buyer's details, item info, and request ID. Finally, it returns the request details along with the seller's contact information so the buyer can reach out.

5.5 System testing

Unit testing is a method used to test small parts of the system, such as individual functions or modules, to make sure they work correctly. Each function is tested separately to check if it gives the expected result.

5.5.1 Unit test

Function	Passed
Register	Yes, and this was achieved through unit tests for the listed functions.
Login	
Creating Post	
Search Items	
Compare Items	
Get Recommendations	
View Post	
Purchase Item	
Edit Post	
Delete Post	
Report Post	
Manage User Account	
Manage User Post	
System Monitoring	

Table 30 Unit Testing

The testing of each function is shown below in which the result of all of them is Pass.

1. Post Flow

No.	Input	Expected Output	Real Output	Comments
1	"post"	Flow initialized, title	"What is the title of your post?"	System successfully starts the post flow

No.	Input	Expected Output	Real Output	Comments
		prompt displayed		

Table 31 Successful Flow Trigger (Post Flow)

No.	Input	Expected Output	Real Output	Comments
1	"Lab coat"	Moves to category step	"Select a category for your post: Lab equipment, Clothing, or Electronics."	MinLength(3) validation passed

Table 32 Successful Title Input (Post Flow)

2. Buy Flow

No.	Input	Expected Output	Real Output	Comments
1	"buy"	Flow initialized, post ID prompt displayed	"Please enter the Post ID you want to purchase:"	System successfully starts the buy flow

Table 33 Successful Flow Trigger (Buy Flow)

No.	Input	Expected Output	Real Output	Comments
1	"15" (not owned post)	Request created, confirmation shown	"Request #1 sent successfully!"	NumberCondition + check_not_own_post (async) passed, finish_action: create_request called

Table 34 Successful Post ID Input (Buy Flow)

3. Compare Flow

No.	Input	Expected Output	Real Output	Comments
1	"compare"	Flow initialized, first post ID prompt displayed	"Enter the first Post ID to compare:"	System successfully starts the compare flow

Table 35 Successful Flow Trigger (Compare Flow)

No.	Input	Expected Output	Real Output	Comments
1	Same ID entered for both posts	Error message or warning shown	Error or warning displayed	Comparing a post with itself – edge case validation

Table 36 Unsuccessful Input – Same Post ID Twice (Compare Flow)

4. Report Flow

No.	Input	Expected Output	Real Output	Comments
1	"report"	Flow initialized, post ID prompt displayed	"Enter the Post ID you want to report:"	System successfully starts the report flow

Table 37 Successful Flow Trigger (Report Flow)

No.	Input	Expected Output	Real Output	Comments
1	"15" (valid, not owned)	Accepted, reason prompt displayed	"Please provide a reason for reporting this post (at least 10 characters):"	NumberCondition + check_not_own_post async passed

Table 38 Successful Post ID Input (Report Flow)

5. Search Flow

No.	Input	Expected Output	Real Output	Comments
1	"search"	Flow initialized, category prompt displayed	"Which category would you like to search? Lab equipment, Clothing, or Electronics."	System successfully starts the search flow

Table 39 Successful Flow Trigger (Search Flow)

No.	Input	Expected Output	Real Output	Comments
1	"Electronics"	Accepted, moves to name step	"What is the name of the item you are looking for?"	ChoiceCondition validation passed

Table 40 Successful Category Input (Search Flow)

6. Edit Post Flow

No.	Input	Expected Output	Real Output	Comments
1	"edit post"	Flow initialized, post ID prompt displayed	"Enter the ID of the post you want to edit:"	System successfully starts the edit post flow

Table 41 Successful Flow Trigger (Edit Post Flow)

No.	Input	Expected Output	Real Output	Comments
1	"1" (owned post)	Accepted, field selection prompt displayed	"What would you like to update? Choose from: Item name, Price, Description, Category, or Image."	NumberCondition + check_own_post async passed

Table 42 Successful Post ID Input (Edit Post Flow)

7. Delete Post Flow

No.	Input	Expected Output	Real Output	Comments
1	"delete post"	Flow initialized, post ID prompt displayed	"Enter the ID of the post you want to delete:"	System successfully starts the delete post flow

Table 43 Successful Flow Trigger (Delete Post Flow)

No.	Input	Expected Output	Real Output	Comments
1	"1" (owned post)	Accepted, confirmation prompt displayed	"Are you sure you want to delete this post? Type 'yes' to confirm or 'cancel' to stop."	NumberCondition + check_own_post async passed

Table 44 Successful Post ID Input (Delete Post Flow)

Chapter 6: Case-study: AI Agent Extension

This chapter documents the architectural extension of the Taibah University Marketplace Bot (TUMB) from a deterministic, rule-based dialogue system to a fully AI-powered conversational agent. The extension introduces a Large Language Model (LLM) agent layer and a Model Context Protocol (MCP) server that together enable natural-language understanding, dynamic tool invocation, and structured data retrieval — all without modifying the existing application logic.

6.1 Motivation and Objectives

The original TUMB app relied on a rule-based intent-matching engine. While effective for a fixed and well-defined set of user requests, such architectures exhibit well-documented limitations in real-world deployment:

- Brittleness to paraphrasing: a user asking "show me cheap lab gear" and a user asking "find affordable laboratory equipment" must both be anticipated explicitly by predefined rules.
- Inability to compose multi-step tasks: a request such as "compare those two laptops and buy the cheaper one" requires multiple sequential actions that rule-based systems cannot chain autonomously.
- High maintenance overhead: every new feature or intent requires manual authoring of rules, regular expression patterns, and response templates.
- No contextual memory within a session beyond what the developer explicitly engineers.

The objectives of this extension were therefore threefold: (1) replace the rule-matching layer with a reasoning LLM capable of understanding natural language in both Arabic and English; (2) expose the existing ActionService business logic as callable tools accessible to the LLM without rewriting any application code; and (3) return structured, renderable data to the frontend alongside the LLM's natural-language reply, preserving the card-based UI experience.

6.2 Overview of Added Technologies

The extension introduces three principal technologies that collaborate to form the upgraded system.

6.2.1 Large Language Models (LLMs)

A Large Language Model is a neural network trained on vast corpora of text, capable of understanding and generating natural language across a wide range of tasks [22]. In this system, Claude Sonnet (claude-sonnet-4-20250514) by Anthropic serves as the reasoning backbone. The model interprets user intent, decides which tools to invoke, formulates the parameters for those tools, and synthesizes a final reply from the tool results. The choice of an instruction-tuned, chat-optimized model ensures that the agent follows complex behavioral guidelines expressed in the system prompt, including response style, purchase confirmation flows, and language-switching behavior.

6.2.2 The Model Context Protocol (MCP)

The Model Context Protocol (MCP) is an open standard introduced by Anthropic in 2024 that defines a uniform interface through which an AI model can discover and invoke external tools or resources. MCP decouples the tool implementation from the model: tools are declared with a name, a natural-language description, and a JSON Schema for their input parameters. The model uses these declarations to reason about which tool to call and with what arguments, without requiring any model-specific integration code.

MCP servers communicate over a transport layer; this implementation uses the stdio (standard input/output) transport, meaning the MCP server process is launched as a child process of the main application and communicates through pipes. This design requires no additional network infrastructure such as HTTP servers or message brokers, simplifying deployment significantly [23].

6.2.3 LangChain and LangGraph

LangChain is an open-source framework for building LLM-powered applications, providing abstractions for models, prompts, tools, and memory [24]. LangGraph, an extension of LangChain, allows developers to define stateful, graph-based agent workflows. This project uses LangGraph's prebuilt `create_react_agent` constructor, which implements the ReAct (Reasoning and Acting) paradigm [25]: at each step, the model reasons about what to do next, takes an action (tool call), observes the result, and continues until it produces a final answer. The `langchain-mcp-adapters` library bridges the MCP tool protocol with LangChain's tool interface, translating MCP tool declarations into LangChain-compatible tool objects.

6.3 System Architecture

The original TUMB system follows a client-server architecture in which the React frontend (client) communicates with a FastAPI backend (server) over HTTP. The AI-agent extension preserves this architecture entirely and introduces two new server-side components — `llm_agent.py` and `mcp_server/server.py` — without altering the client or any existing server module. Figure 41 illustrates the updated client-server architecture.

Side	Component	Role in the Architecture
Client	React Frontend	Sends HTTP requests containing the user message and full conversation history; renders the text reply and tool-result UI cards returned by the server
Server	FastAPI Router	Receives HTTP requests; authenticates the user; forwards <code>user_id</code> , name, and message history to the LLM agent; returns the agent response as JSON
Server (NEW)	<code>llm_agent.py</code>	Singleton agent process; manages the ReAct reasoning loop; calls MCP tools; extracts structured tool data for card rendering
Server (NEW)	<code>mcp_server/server.py</code>	Child subprocess launched by the agent; exposes ten MCP tools; dispatches each tool call to the existing <code>ActionService</code>
Server	<code>ActionService</code>	Core business logic; processes all marketplace operations; unchanged from the original system
Server	Repositories + PostgreSQL	Persistence layer; accessed exclusively through <code>ActionService</code> ; unchanged

Table 45 Updated client-server architecture after the AI-agent extension

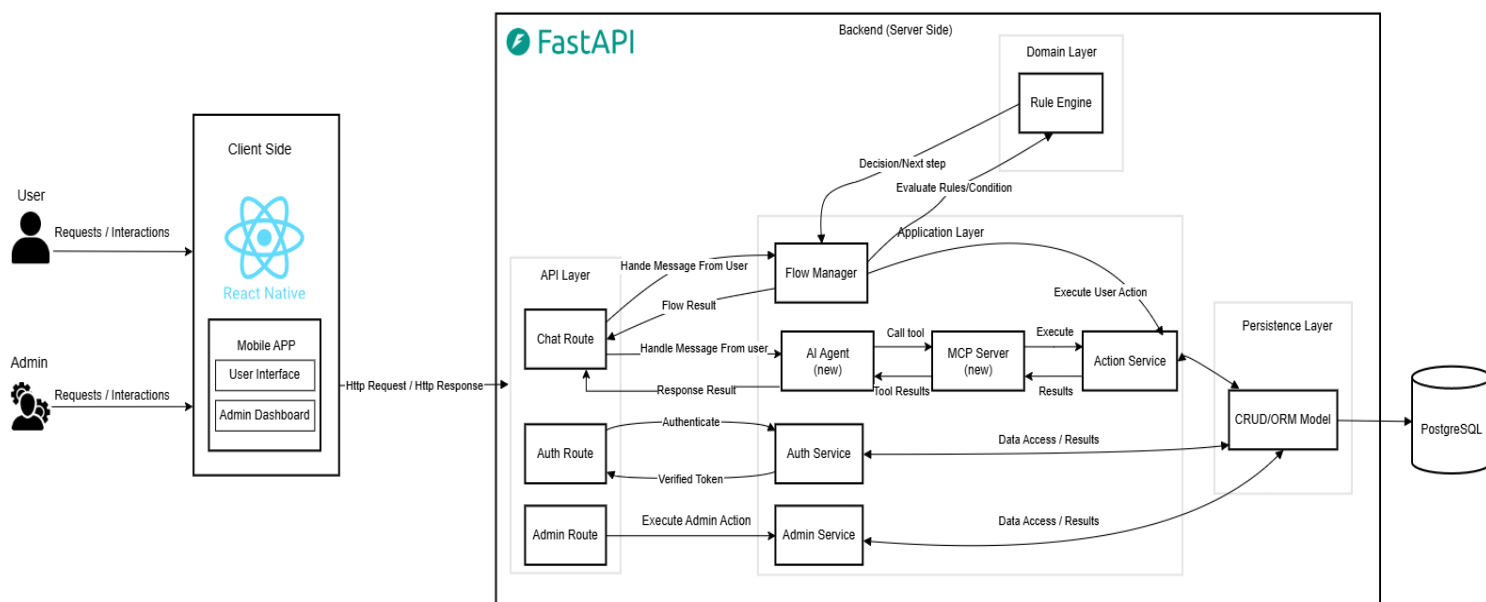


Figure 41 Client-server architecture of the AI-powered TUMB system

The client remains unaware of whether a request is handled by the rule-based engine or the AI agent: it sends the same message payload and receives the same JSON structure in both cases. All new complexity is confined to the server side, which is a fundamental property of the client-server model. The MCP server is not a separate networked service; it runs as a stdio subprocess within the same server process, communicating with the agent through pipes rather than HTTP.

A critical design decision was to keep the two new components strictly additive. The LLM agent and MCP server wrap the existing ActionService rather than replacing it, meaning the original rule-based chatbot can remain operational in parallel and both modes share identical business-logic guarantees.

6.4 MCP Server Implementation

The MCP server (`mcp_server/server.py`) is the bridge between the AI agent and the application's data and business logic. It is launched automatically as a subprocess by the LLM agent at application startup and must not be executed independently.

6.4.1 Tool Declaration

The server exposes ten tools through the `@server.list_tools()` handler. Each tool is declared as an `mcp.types.Tool` object with three components: a machine-readable name, a natural-language description that guides the LLM's tool-selection reasoning, and a JSON

Schema that specifies the required and optional input parameters. Table 47 summarizes the ten tools:

Tool Name	Description	Key Parameters
search_posts	Search marketplace listings by keyword and/or category	item_name, category, cursor, user_id
create_post	Create a new item listing	item_name, category, price, description, quantity, user_id
buy_post	Send a purchase request to a seller	post_id, quantity, user_id
compare_posts	Compare two listings and recommend best value	post_id_1, post_id_2, user_id
report_post	Report a listing for a policy violation	post_id, reason, user_id
update_post	Update fields of the user's own listing	post_id, [optional fields], user_id
delete_post	Permanently delete the user's own listing	post_id, user_id
get_my_posts	List all active listings by the current user	cursor, user_id
get_my_requests	List all purchase requests sent by the user	user_id
get_recommendations	Return personalised listing recommendations	user_id

Table 46 MCP tools exposed by the TUMB MCP Server

```

49 @server.list_tools()
50 async def list_tools() -> list[mcp.types.Tool]:
51     return [
52
53         types.Tool(
54             name="search_posts",
55             description=(
56                 "Search marketplace listings by item name and/or category. "
57                 "Returns up to 4 results with cursor-based pagination. "
58                 "Pass 'skip' as item_name to search by category only."
59             ),
60             inputSchema={
61                 "type": "object",
62                 "properties": {
63                     "item_name": {"type": "string", "description": "Keyword to search. Use 'skip' to ignore."},
64                     "category": {"type": "string", "description": "lab equipment | clothing | electronics"},
65                     "cursor": {"type": "integer", "description": "Last post ID from previous page for pagination"},
66                     "user_id": {"type": "integer"},
67                 },
68                 "required": ["user_id"],
69             },
70         ),
71     ]

```

Figure 42 Tool declaration using `mcp.types`

The natural-language descriptions in each tool declaration are critical: they are passed verbatim to the LLM and form the primary basis on which the model decides which tool to call. For example, the `search_posts` description explicitly instructs the model to pass 'skip' as `item_name` when searching by category only, eliminating an ambiguity that would otherwise produce incorrect tool calls.

6.4.2 Tool Dispatch

When the LLM invokes a tool, the MCP server's `@server.call_tool()` handler receives the tool name and a dictionary of arguments. A dedicated `_dispatch()` function routes each tool call to the corresponding `ActionService` method by constructing an `ActionContext` object — the same data-transfer object used by the original rule-based system — and calling `service.execute()` with the appropriate action string. This routing approach means that:

- All existing input validation, business rules, and error handling inside `ActionService` remain active.
- The AI agent cannot bypass any application-layer safeguards by virtue of being AI-powered.
- Adding a new action to the system requires only adding a new tool declaration in the MCP server; no changes to `ActionService` are required.

Each tool call creates a fresh database session via `AsyncSessionLocal`, ensuring proper transaction isolation between concurrent requests. The session is closed in a `finally` block regardless of whether the tool call succeeded or raised an exception. Errors from `ActionService` (of type `ActionError`) are caught and serialized as JSON objects containing an `"error"` key. This allows the LLM to receive a structured error description and relay it to the user in natural language rather than exposing a raw exception trace.

```

205 @server.call_tool()
206 async def call_tool(name: str, arguments: dict) -> list[types.TextContent]:
207     service, session = await _make_service()
208     user_id = int(arguments.get("user_id", 0))
209
210     try:
211         result = await _dispatch(name, arguments, user_id, service)
212     except ActionError as e:
213         result = {"error": e.message}
214     except Exception as e:
215         result = {"error": str(e)}
216     finally:
217         await session.close()
218
219     return [types.TextContent(type="text", text=json.dumps(result, default=str))]
220
221
222 async def _dispatch(name: str, args: dict, user_id: int, service: ActionService) -> dict:
223     """Route tool name -> ActionService call."""
224
225     if name == "search_posts":
226         ctx = ActionContext(user_id=user_id, data={
227             "item_name": args.get("item_name"),
228             "category": args.get("category"),
229             "last_id": args.get("cursor"),
230         })
231         return (await service.execute("search_post", ctx))["action_data"]
232
233     elif name == "create_post":
234         ctx = ActionContext(user_id=user_id, data={
235             "item_name": args["item_name"],
236             "category": args["category"],
237             "price": args["price"],

```

Figure 43 Tool dispatch logic

6.5 LLM Agent Implementation

6.5.1 Initialization

The agent is initialized once at application startup via the `init_agent()` coroutine and is subsequently reused for every incoming request. Initialization proceeds in two steps. First, a `MultiServerMCPClient` is instantiated and configured to launch the MCP server as a child subprocess using the same Python interpreter as the host process. The client establishes a stdio transport connection and retrieves the tool declarations from the server via the MCP handshake protocol. Second, the tool list is passed to LangGraph's `create_react_agent` constructor alongside a `ChatAnthropic` model instance, producing a

compiled agent graph. The agent graph is stored as a module-level singleton, making per-request invocations lightweight.

```
97 async def init_agent():
98     """
99     Called once when the app starts.
100     Connects to the MCP server, loads the tools, and builds the agent.
101     """
102     global _agent, _tools
103
104     # Connect to the MCP server subprocess and get its tools as LangChain tools
105     mcp_client = MultiServerMCPClient(
106         {
107             "tumb": {
108                 "command": sys.executable,          # the same Python that's running the app
109                 "args": [MCP_SERVER_PATH],
110                 "transport": "stdio",               # MCP communicates over stdin/stdout pipes
111                 "env": os.environ.copy(),           # pass all env vars (DATABASE_URL etc.)
112             }
113         }
114     )
115     _tools = await mcp_client.get_tools()
116
117     # Build the ReAct agent: model + tools
118     # create_react_agent handles the loop: call model → run tools → call model → ...
119     model = ChatAnthropic(model="claude-sonnet-4-20250514", max_tokens=1024)
120     _agent = create_react_agent(model=model, tools=_tools)
121
122
```

Figure 44 Agent initialization

The system prompt is the primary mechanism for constraining and directing the LLM's behavior. It is a string template formatted at request time with the authenticated user's database ID and display name. This per-request personalization ensures the agent never needs to ask for identity information that the backend already possesses, and it prevents identity confusion in multi-user environments.

The system prompt encodes the following behavioral policies:

- Tool-first policy: the agent must always call a tool when the user asks about marketplace data, rather than fabricating an answer from training knowledge.
- No hallucination of identifiers: post IDs, prices, and item names must only be used if returned by a tool.
- Category constraint: only the three valid categories ("lab equipment", "clothing", "electronics") may be passed to tools.
- Bilingual behavior: the agent responds in whichever language (Arabic or English) the user writes in.
- Typo correction: the agent corrects obvious spelling errors before passing parameters to tools (e.g., "labtops" becomes "laptops").

- Purchase confirmation gate: the agent is explicitly forbidden from invoking `buy_post` until the user has replied with an explicit affirmative in a separate turn, preventing accidental purchases.
- Plain-text output style: markdown formatting is prohibited in all responses to ensure compatibility with the frontend's chat renderer.

The explicit purchase-confirmation constraint illustrates a key principle of prompt engineering for agentic systems: safety-critical actions must be gated behind explicit user intent verification, not inferred from a single ambiguous utterance.

```

32 SYSTEM_PROMPT = """\
33 You are TUMB Assistant – a friendly AI helper for the Taibah University Marketplace.
34 You help Taibah University students buy, sell, and manage academic items on campus.
35
36 The current user's ID is {user_id} and their name is {name}.
37
38 What you can do (via tools):
39 - Search listings by name or category
40 - Create a new listing to sell an item
41 - Send a purchase request to a seller
42 - Compare two listings and recommend the best value
43 - Report an inappropriate listing
44 - Update or delete the user's own listing
45 - Show the user's own listings and purchase requests
46 - Show personalised recommendations
47
48 Important rules:
49 - Always call a tool when the user asks about listings or marketplace data.
50 - Never make up post IDs, prices, or item names – only use what tools return.
51 - Valid categories are: "lab equipment", "clothing", "electronics".
52 - Prices are in Saudi Riyals (SAR).
53 - Respond in the same language the user writes in (Arabic or English).
54 - You already know the user's ID – never ask them for it.
55 - Automatically correct spelling mistakes or typos in user requests before passing parameters to tools (e.g., if the user asks for "
56 - App Explanation: If the user asks what the app does, what it offers, or how you can help, explain that TUMB (Taibah University Mar
57
58 Purchase Flow & Tool Examples (CRITICAL):
59 1. If a user wants to buy an item, you MUST first know the exact post ID. If you don't know it, use the search tool first.
60 2. BEFORE calling the buy_post tool, you MUST ask the user to explicitly confirm the purchase by showing them the item name and post
61 3. When asking for this confirmation, DO NOT call any tools. Just reply with plain text. This ensures no unexpected cards are shown
62 4. ABSOLUTE STOP: You are FORBIDDEN from calling the buy_post tool until the user has explicitly replied with "yes", "confirm", or s
63

```

Figure 45 System prompt template in `llm_agent.py`

At request time, the `run_agent()` function constructs a message list consisting of the formatted system prompt followed by the full conversation history converted to LangChain message objects. This stateless-per-call design means the agent has no persistent memory; all context is re-supplied on every call from the client-managed history. The agent is then invoked via `_agent.ainvoke()`, which executes the following loop asynchronously:

1. The model receives all messages and generates a response. If the response contains tool-use instructions, LangGraph intercepts them before returning to the caller.
2. LangGraph serialises each tool call and sends it to the MCP server over the stdio transport.

3. The MCP server dispatches the call to ActionService, serialises the result as JSON, and returns it to the agent.
4. LangGraph appends the tool result as a ToolMessage to the message list and re-invokes the model.
5. The model generates a natural-language reply synthesising the tool result. If no further tool calls are needed, the loop terminates.
6. The final AIMessage content is extracted as the text reply, and the structured tool data is extracted separately for frontend card rendering.

```

125 async def run_agent(user_id: int, name: str, messages: list[dict[str, Any]]) -> dict:
126
127     Run one conversation turn through the agent.
128
129     Parameters
130     -----
131     user_id : the logged-in user's database ID (injected into the system prompt)
132     name    : the user's display name
133     messages : full conversation history from the client
134               (the last item is the new user message)
135
136     Returns
137     -----
138     A dict with:
139     - "reply": the assistant's text reply
140     - "data": structured tool data for card rendering (or None)
141     """
142
143     # Build the message list: system prompt + conversation history
144     all_messages = [
145         SystemMessage(content=SYSTEM_PROMPT.format(user_id=user_id, name=name)),
146         *_history_to_langchain(messages),
147     ]
148
149     # Run the agent – LangGraph handles the tool-calling loop automatically
150     result = await _agent.ainvoke({"messages": all_messages})
151
152     # The last message in the result is always the final assistant reply
153     reply = result["messages"][-1].content
154     if not isinstance(reply, str):
155         reply = str(reply)
156
157     # Extract structured data from any tool calls the agent made
158     tool_data = _extract_tool_data(result["messages"])
159
160     # Save the assistant reply into the history so the client can send it next time
161     messages.append({"role": "assistant", "content": reply})
162     return {"reply": reply, "data": tool_data}

```

Figure 46 The run_agent() entry point

The original chatbot returned structured action_data objects that the React frontend used to render rich UI cards (listing cards, comparison views, purchase request summaries). To preserve this behaviour, the _extract_tool_data() function scans the agent's final message list for ToolMessage objects and extracts the last successful tool result. It then maps the MCP tool name to the frontend's expected action_name string using the TOOL_TO_ACTION dictionary, producing a response payload of the form {"action_name": "...", "action_data": {...}}.

This mapping layer decouples the MCP tool naming convention from the frontend's existing card-rendering logic, allowing both systems to evolve independently without coupling changes [26].

```

77  TOOL_TO_ACTION = {
78      "search_posts":      "search_post",
79      "get_my_posts":      "user_posts",
80      "get_recommendations": "user_recommendations",
81      "compare_posts":     "compare_posts",
82      "buy_post":          "create_request",
83      "get_my_requests":   "user_requests",
84      "report_post":       "report_post",
85      "create_post":       "create_post",
86      "update_post":       "update_post",
87      "delete_post":       "delete_post",
88  }
89
167 def _extract_tool_data(messages) -> dict | None:
168     """
169     Scan the agent's message list for ToolMessage objects and extract the
170     last successful tool result that maps to a displayable card type.
171     Returns a dict like {"action_name": "search_post", "action_data": {...}}
172     or None if no card-worthy data was found.
173     """
174     last_tool_name = None
175     last_tool_result = None
176
177     for msg in messages:
178         if not isinstance(msg, ToolMessage):
179             continue
180
181         # ToolMessage.content may be a plain JSON string or a list of content blocks
182         raw = msg.content
183         if isinstance(raw, list):
184             raw = " ".join(
185                 block.get("text", "") if isinstance(block, dict) else str(block)
186                 for block in raw
187             )
188
189         try:
190             parsed = json.loads(raw)
191         except (json.JSONDecodeError, TypeError):
192             continue
193
194         if isinstance(parsed, dict) and "error" not in parsed:
195             last_tool_name = msg.name
196             last_tool_result = parsed
197
198     if not last_tool_name or not last_tool_result:
199         return None
200
201     action_name = TOOL_TO_ACTION.get(last_tool_name)
202     if not action_name:
203         return None
204
205     return {
206         "action_name": action_name,
207         "action_data": last_tool_result,
208     }
209
210

```

Figure 47 Tool-to-action mapping and structured data

6.6 Conversation Flow Walkthrough

To illustrate how the components interact in practice, consider the user request: "I'm looking for a used calculator, can you find one for me?" The following sequence describes the complete system behavior for this single turn:

7. The FastAPI router authenticates the request, retrieves the user's ID and name, and calls `run_agent(user_id=42, name="Ammar", messages=[...])`.
8. `run_agent` prepends the formatted system prompt to the conversation history and calls `_agent.ainvoke()`.
9. The LLM reasons that the user wants to search listings. It selects the `search_posts` tool and supplies `{"item_name": "calculator", "user_id": 42}` as arguments.
10. LangGraph sends the tool call to the MCP server. The server constructs an `ActionContext` and calls `service.execute("search_post", ctx)`, which queries the database and returns matching posts.
11. The MCP server serializes the result as JSON and returns it to the agent via the `stdio` pipe.
12. The LLM receives the tool result and generates the reply: "Found a couple of calculators for you! Check them out below."
13. `_extract_tool_data` identifies the `search_posts` `ToolMessage`, maps it to `action_name="search_post"`, and packages the structured listing data.
14. The API endpoint returns `{"reply": "Found a couple...", "data": {"action_name": "search_post", "action_data": {...}}}` to the front end.
15. The React frontend renders the text reply in the chat bubble and displays the listing cards below it.

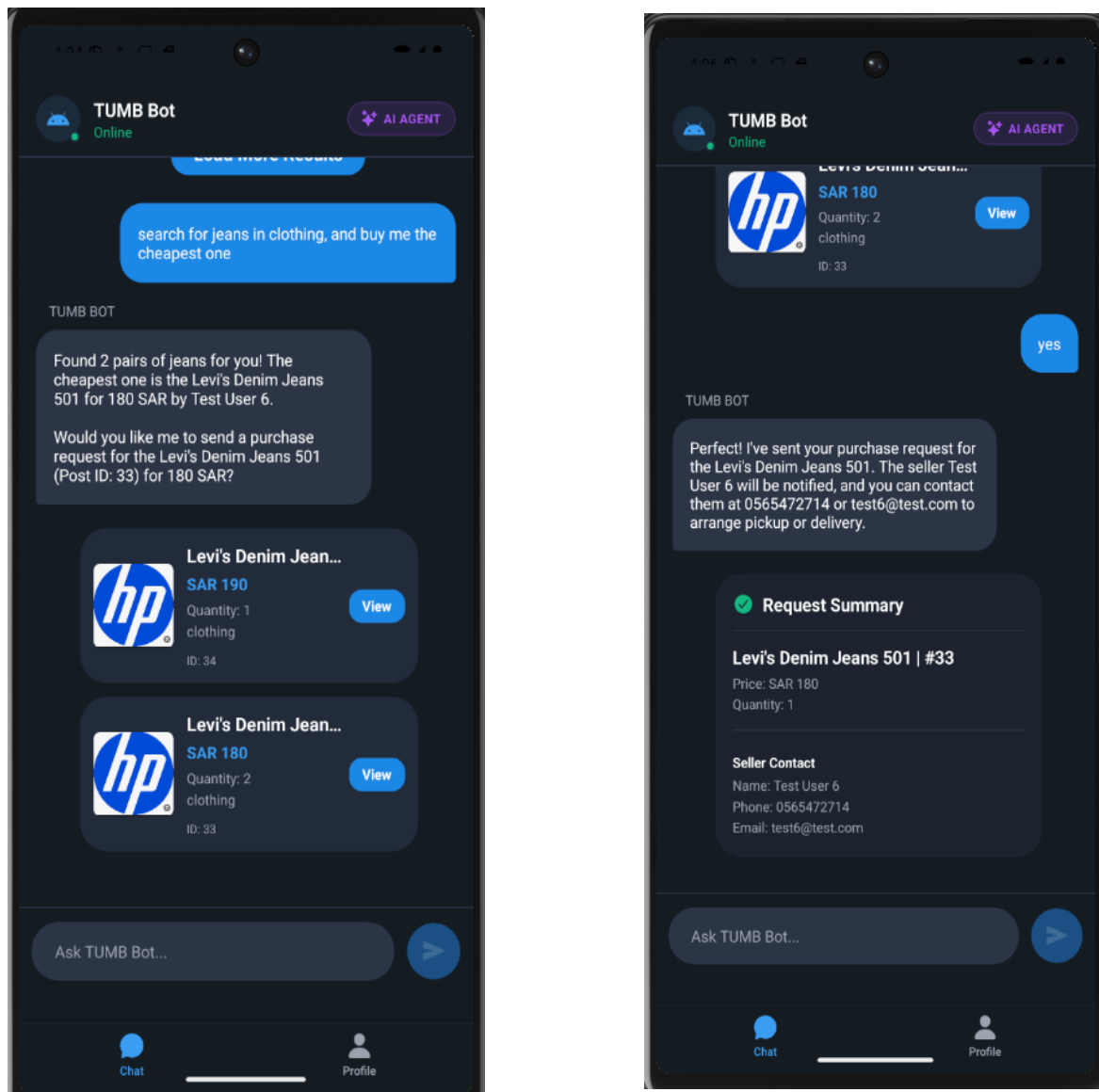


Figure 48 Live conversation demonstrating the full request-response cycle

6.7 Comparative Analysis: Rule-Based vs. AI-Powered Agent

Dimension	Rule-Based System	AI-Powered Agent
Intent understanding	Exact keyword / regex matching	Natural language understanding via LLM
Paraphrase handling	Fails unless all variants are predefined	Handles arbitrary paraphrasing and typos
Multi-step tasks	Single intent per turn only	Agent loops autonomously over multiple tool calls

Dimension	Rule-Based System	AI-Powered Agent
Bilingual support	Requires explicit Arabic/English rule sets	Language-switching is handled natively by LLM
Maintenance	High: every new feature requires new rules	Low: new features require only a new MCP tool
Predictability	Fully deterministic	Non-deterministic; response wording varies
Latency	Sub-millisecond	1 s to 3 s (LLM API round trip)
Cost	Zero inference cost	Incurs API usage cost per request
Safety controls	Encoded in rule logic	Encoded in system prompt; requires careful engineering
Structured output	Always returned	Returned only when a tool is called

Table 47 Comparative analysis of the rule-based and AI-powered agent approaches

6.8 Advantages of the AI-Powered Approach

6.8.1 Robustness to Linguistic Variation

Because the LLM was pre-trained on large-scale corpora encompassing a wide range of linguistic expressions, it correctly interprets semantically equivalent requests expressed in different vocabulary, sentence structures, or languages. A request phrased in Arabic — for instance, asking for a cheap laptop — is processed correctly without any language-specific configuration, enabling a genuinely bilingual user experience.

6.8.2 Autonomous Multi-Step Reasoning

The ReAct loop enables the agent to decompose complex user goals into a sequence of tool calls executed without additional input from the user. For example, the request "compare the two cheapest laptops" requires first searching for laptops, identifying the

two lowest-priced results, and then calling the `compare_posts` tool — three logical steps that the agent handles within a single conversation turn.

6.8.3 Reduced Development Effort for New Features

Adding a new capability to the rule-based system required authoring new intents, training data, and response templates. In the agent architecture, exposing a new `ActionService` method requires only writing a new Tool declaration in the MCP server. The LLM will immediately understand how to use it based on the declaration's natural-language description, with no additional training or configuration.

6.8.4 Backward Compatibility

Because the MCP server maps directly onto the existing `ActionService` interface, the upgrade does not break any existing functionality. The card-rendering frontend continues to receive the same `action_name` and `action_data` structure it expected from the rule-based system, requiring no frontend changes.

6.8.5 Standards-Based Tool Integration

Using MCP as the tool interface rather than a proprietary function-calling mechanism means the agent is not locked to any specific LLM provider. Any MCP-compatible model could replace the current Claude Sonnet backend with minimal code changes, improving long-term maintainability and vendor flexibility.

6.9 Limitations and Challenges

6.9.1 Latency

Each conversation turn requires at least one synchronous round trip to the Anthropic API, introducing a latency of approximately 1 second to several seconds depending on network conditions and model load. Multi-step tasks that require multiple tool calls incur additional latency for each model invocation in the ReAct loop. This is a fundamental constraint of remote LLM inference that cannot be eliminated without on-premise model deployment.

6.9.2 Operational Cost

Unlike the rule-based system, which incurs zero marginal inference cost, the AI agent consumes API credits proportional to the number of input and output tokens processed per request. For a university marketplace with moderate traffic, this cost is manageable; however, it becomes a significant consideration at scale, and budget limits or rate throttling may be necessary in production deployment.

6.9.3 Non-Determinism

LLMs are probabilistic systems: two identical inputs may produce different outputs. While the system prompt constrains the agent's behavior substantially, edge cases may produce unexpected responses. Comprehensive prompt engineering, regression testing, and systematic evaluation of the system prompt are required to achieve reliable behavioral consistency across all user scenarios.

6.9.4 Prompt Injection and Adversarial Inputs

Because the LLM processes raw user text, malicious users may attempt prompt injection attacks — crafting inputs designed to override the system prompt's instructions and cause the agent to behave in unintended ways (e.g., calling `delete_post` without authorization). The current implementation relies on the model's instruction-following robustness as the primary defense. Production deployments should supplement this with input sanitization, output validation, and action-level authorization checks independent of the agent's decisions.

6.9.5 Statelessness and Context Window Limits

The agent has no persistent memory; the full conversation history is re-submitted on every turn. As conversations grow longer, the token count approaches the model's context window limit, at which point earlier parts of the history must be truncated. In the current implementation, history management is the responsibility of the client, which introduces a risk of context loss in very long sessions.

6.10 Security Considerations

Several security properties of the implementation are worth highlighting explicitly:

- **User ID injection:** the authenticated user's ID is injected into the system prompt by the server, not read from user input. This ensures that a user cannot claim to be another user to trigger actions on their behalf.
- **Authorization enforcement:** tool calls that require ownership (`update_post`, `delete_post`) are validated by `ActionService`, not by the agent. The agent cannot bypass these checks even if instructed to do so by a malicious input.
- **No raw database access:** the MCP server has no direct SQL access; it calls `ActionService` methods exclusively. This preserves the application-layer boundary and ensures all business rules are applied consistently.
- **Purchase confirmation gate:** the explicit two-turn confirmation requirement for `buy_post` prevents unintended purchases that might otherwise result from ambiguous user utterances.

6.11 Summary

This chapter presented the design and implementation of an AI-powered agent extension to the TUMB app. The extension replaces the rule-based dialogue layer with a LangGraph ReAct agent backed by Claude Sonnet, while exposing the existing application's business logic as a set of MCP tools. The architecture is strictly additive — no existing code was modified — and the frontend continues to receive the same structured data it depended upon in the original system.

The resulting system is significantly more capable than its predecessor: it handles natural language in Arabic and English, corrects typographical errors, chains multiple tool calls autonomously, and provides a conversational experience that is substantially more flexible and user-friendly. These gains come at the cost of increased latency, operational API expenses, and reduced output determinism — trade-offs that are acceptable for an academic project and manageable in production with appropriate engineering mitigations. The use of MCP as the tool interface represents a forward-looking architectural choice: it positions the system for straightforward integration with future LLM providers and MCP-compatible tooling ecosystems, and it establishes a clean, standards-based boundary between the AI reasoning layer and the application business logic.

Chapter 7: Conclusion

The Taibah University Marketplace Bot (TUMB) application was successfully developed to address the financial and logistical challenges faced by the university community in acquiring academic supplies. By providing a centralized, secure, and university-exclusive platform, TUMB facilitates the reuse of resources, supporting both economic efficiency and environmental sustainability within a circular economy framework.

In addition to its core functionality, the project explored the integration of a Large Language Model (LLM)–based extension to enhance the conversational experience. While this extension demonstrated promising capabilities, transitioning it into the primary interaction mechanism—replacing the rule-based chatbot—requires extensive testing and validation to ensure reliability, accuracy, and system stability.

The core implementation of this project is centered on a rule-based conversational interface, which simplifies complex e-commerce interactions—such as searching, comparing, and posting items—into intuitive, guided dialogues. Through a structured Waterfall development methodology, the team ensured the delivery of a robust system comprising a React Native mobile application, a FastAPI backend, and a PostgreSQL database, all working seamlessly to provide a trusted peer-to-peer trading environment.

7.1 Goals Achieved

The project successfully achieved its primary objectives by implementing a functional and user-friendly marketplace platform tailored specifically for Taibah University members. One of the key accomplishments is the implementation of a rule-based conversational chatbot that guides users step-by-step through marketplace operations, significantly reducing complexity and improving user experience.

Additionally, the system introduced an integrated item comparison feature that allows users to evaluate two products side-by-side directly within the chat interface, enabling better decision-making. The platform also ensures a secure and exclusive environment by restricting access to verified university email accounts, which minimizes fraud and enhances trust among users.

Furthermore, an automated email notification system was implemented to instantly inform sellers when a buyer expresses interest, thereby streamlining communication without the need for external messaging platforms.

7.2 Limitations and Future Work

Despite its successful implementation, the TUMB system has several limitations that can be addressed in future work. One of the main limitations is the reliance on a rule-based chatbot, which may struggle to handle complex or unexpected user queries. Although an LLM-based extension was developed, it has not yet been adopted as the primary interaction method due to the need for further testing, performance evaluation, and reliability assurance.

Another challenge encountered during development was the restriction imposed by the university email system, which blocks messages from unknown external domains. This limitation prevented the successful delivery of One-Time Password (OTP) verification emails to university accounts, affecting the user authentication process.

Additionally, the system currently lacks integrated payment functionality, requiring users to complete transactions outside the application. Moreover, testing was conducted using limited or simulated data, which may not fully reflect real-world usage scenarios.

To enhance the system, future work could focus on further refining and optimizing the integrated Artificial Intelligence and Natural Language Processing (NLP) components to improve accuracy, reliability, and overall user experience. Another potential improvement is the incorporation of secure local payment gateways such as Mada, along with on-campus logistics solutions like smart lockers to facilitate safe item exchange.

Further enhancements may include developing advanced analytics tools to track user behavior and providing personalized recommendations, as well as supporting multi-branch operations to enable efficient transactions across different university campuses.

References

- [1] P. Kotler and K. Keller, *Marketing Management*, 16th ed., Pearson Education, 2022.
- [2] J. Nielsen, “Usability Engineering,” Morgan Kaufmann, 1994.
- [3] W3C, “Web Content Accessibility Guidelines (WCAG) 2.2,” World Wide Web Consortium, 2023.
- [4] M. H. Al-Qirim, “Secure Payment Gateway Architectures for E- Commerce Applications,” *IEEE Access*, vol. 8, pp. 12345–12358, 2020.
- [5] A. Raman, “E-Commerce Logistics: Strategies for Last-Mile Delivery,” *Harvard Business Review*, 2021.
- [6] M. D. Felder and E. Hayes-Roth, “Rule-Based Expert Systems,” Springer, 2019
- [7] D. Jurafsky and J. Martin, “Speech and Language Processing,” Pearson, 2023.
- [8] S. Young, “Statistical Dialogue Systems: A Review,” *IEEE Signal Processing Magazine*, vol. 27, no. 3, pp. 140–149, 2010.
- [9] B. Webber, “Response Generation in Natural Language Systems,” *ACM Transactions on Information Systems*, vol. 38, no. 3, pp. 1–25, 2020.
- [10] M. Henderson, “Context Tracking in Conversational Agents,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 22, no. 4, pp. 727–737, 2014.
- [11] R. Botsman and R. Rogers, *What’s Mine Is Yours: The Rise of Collaborative Consumption*, Harper Business, 2010.
- [12] Ellen MacArthur Foundation, *Circular Economy Principles*, 2022. [Online]. Available: <https://ellenmacarthurfoundation.org>
- [13] M. F. McTear, *Conversational AI: Dialogue Systems, Conversational Agents, and Chatbots*, Springer, 2020.
- [14] B. A. Shawar and E. Atwell, “Chatbots: Are They Really Useful?,” *LDV Forum*, vol. 22, no. 1, pp. 29–49, 2007.
- [15] I. Sommerville, *Software Engineering*, 11th ed., Addison-Wesley, 2015.
- [16] MDN Web Docs, “REST API Basics,” 2024. [Online]. Available: <https://developer.mozilla.org>
- [17] Google Developers, “Flutter Documentation,” 2024. [Online]. Available: <https://flutter.dev/docs>
- [18] Haraaj, <https://haraaj.com.sa>

- [19] Facebook Marketplace, <https://www.facebook.com/marketplace/>
- [20] TreasureX, <https://thetreasurex.com>
- [21] KFUPM store, <https://kfupmstore.com>
- [22] Brown, T. B., et al. Language Models are Few-Shot Learners. Advances in Neural Information Processing Systems, 33, 1877–1901, 2020.
- [23] Anthropic. Model Context Protocol (MCP) Specification, 2024. Retrieved from <https://modelcontextprotocol.io>
- [24] Chase, H. LangChain [Software library], 2022. Retrieved from <https://github.com/langchain-ai/langchain>
- [25] Yao, S., et al. ReAct: Synergizing Reasoning and Acting in Language Models. Proceedings of the International Conference on Learning Representations (ICLR 2023).
- [26] LangGraph Documentation. create_react_agent. LangChain AI, 2024. Retrieved from <https://langchain-ai.github.io/langgraph>.